iToken: One-Time-Use Anonymous Token with Issuance Hiding

Zengpeng Li[1], Xiangyu Su[2], Dongfang Wei[1], Guangyu Liao[3], and Mei Wang[1]

[1] School of Cyber Science and Technology, Shandong University
[2] Institute of Science Tokyo
[3] Harbin Engineering University

Abstract. Privacy-Enhancing Know Your Customer (KYC) integrates one-time-use anonymous tokens (OTATs) into self-sovereign identity frameworks, such as the EU Digital Identity (EUDI) Wallet, Apple's Private Access Tokens, and W3C's Privacy-Preserving Advertising proposals (*e.g.*, Private State Tokens), to enable regulatory compliance while preserving user anonymity. To mitigate targeted denial-of-service (DoS) attacks and prevent token misuse (*e.g.*, farming and replay), this paper designs a new OTAT, iToken, that first achieves issuer hiding not only at verification but also throughout issuance, thereby strengthening both OTAT's resilience and user privacy.

We introduce a new primitive, a canonical blind ring signature (BRS), that adopts a blind-and-ring pattern, ensuring the ring structure is present from the outset and is initiated by the signer within the interactive blind signing protocol. We also provide two generic constructions, one from a linear function (LF) and homomorphic encryption, and another from an LF and a commit-and-prove sum argument. We finally prototype BRS and iToken, achieving efficient signing bandwidth and competitive computational performance.

Keywords: Anonymous Token, Blind Ring Signature, Authorization

1 Introduction

A one-time-use anonymous token (OTAT) can indeed be viewed as a specialized, lightweight instance of anonymous credentials, since only one attribute is signed, as depicted in Figure 1. Real-world OTAT applications are deployed incrementally, such as Privacy Pass [20] for anonymous authentication (used by Cloudflare, Google, and Apple) and Payment tokens (*e.g.*, Apple Pay's DPAN). While anonymous tokens and credentials (*e.g.*, in EUDI Wallet and mDLs) offer strong privacy in theory, real-world deployments have exposed practical vulnerabilities, side-channel risks, and design trade-offs. There are issuer centralization risks and token-specific attacks examples: (i) single–point correlation, *i.e.*, even if tokens/credentials are anonymous, the issuer knows who got which token/credential; (ii) weak issuance protocols, *i.e.*, some pilots use non-blind issuance (*e.g.*, plain JWTs signed by issuer), breaking unlinkability from the start; (iii) token reuse (double-spending), *i.e.*, if token redemption isn't atomically tied to consumption (*e.g.*, no trusted ledger or server-side dedup), tokens can be reused in *Privacy-Pass*, fixed in IETF RFC9494; (iv) and token farming, *i.e.*, attackers create many fake identities (*e.g.*, burner phones and temporary eIDs) and collect many tokens, then use these tokens to bypass rate limits or gatekeeping (*e.g.*, "verified user" forum access).

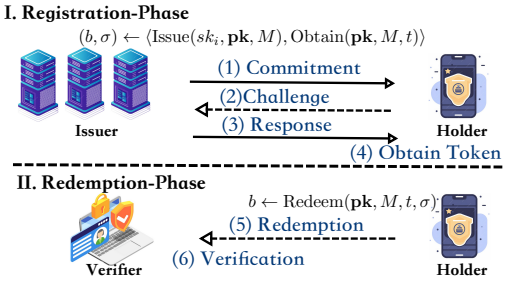


Fig. 1: Illustration of OTAT Protocol

Privacy Pass (the most iconic OTAT) can be deployed alongside authorization systems such as OAuth 2.0 [20], and it can significantly improve token unlinkability and attribute blindness, helping to mitigate risks such as token farming and cross-context tracking [36,51,17,2,28]. To our knowledge, there is only one concrete OTAT

instantiation with issuer hiding using Structure-Preserving Signatures of Equivalence Classes (SPS-EQ) [53], and no further OTAT solutions for hiding issuers. Additionally, Bosk *et al.* [9], Sanders-Traoŕ [47], Rosenberg *et al.* [46], and Katz-Sefranek [34] provided the concrete issuer-hiding anonymous credential instantiations. However, existing issuer-hiding solutions hide the issuer only during verification, not during issuance. As a result, adversaries may still conduct surveillance and covert data collection by leveraging issuer-based profiling (*e.g.*, analyzing ZK proof transcripts, timestamps, and IP addresses), thereby enabling long-term reconstruction of users' activity patterns.

To mitigate token misuse, a natural approach is to ensure issuer anonymity while maintaining user privacy. This motivates the following question:

Is it possible to achieve issuer anonymity during the token issuance (not just at redemption)?

The answer is affirmative: we design an OTAT scheme that hides the issuer during both issuance and redemption while preserving blind token issuance. Below, we summarize our main contributions and provide overviews of the techniques.

1.1 Contributions

- *Canonical Blind Ring Signature.* To render the signer unobservable (even indistinguishable) while preserving blindness, we revisit a legacy yet comparatively underexplored cryptographic primitive: the *blind ring signature* (BRS). We develop a modern formalization that explicitly addresses concurrent security. We then present two generic constructions and provide a rigorous analysis of their correctness, unforgeability, signer-anonymity, and blindness.
- *OTAT with Issuance Hiding.* We present a new OTAT scheme (named iToken) to mitigate token misuse. iToken is built upon our designed BRS, in which multiple issuers form a resilient, collective ring. iToken achieves issuer anonymity during token issuance, not merely at verification, and effectively prevents single-point correlation and collusion among issuers. The security properties of one-more unforgeability, issuance hiding, and unlinkability are all rigorously guaranteed by the corresponding properties of the underlying BRS.
- *Proof of Concept Performance Evaluation.* We implement a prototype of BRS [1] and conduct a comprehensive evaluation at two levels: microbenchmarks of the cryptographic BRS primitive within distributed environments with up to 32 ring members; and macrobenchmarks of the iToken protocol compared with existing OTATs. The performance of BRS demonstrates that our proposed iToken scheme achieves better computational and communication efficiency than state-of-the-art issuer-hiding alternatives.

1.2 Technical Overview

Most existing OTAT work focuses on user privacy, which requires that no malicious verifiers or issuers, even when colluding, can learn a user's private attribute (attribute blindness) or link the user/token to a specific issuance session (token unlinkability). The token misuse and targeted DoS attacks described above highlight the need for issuer-side privacy. Hence, we introduce the notion of *issuance hiding*, which requires that no malicious users or verifiers can determine which authority issued a given token, even if the adversary participates in the issuance session. A related issuer-hiding notion, concerning only the redemption (verification) phase, appears in the broader anonymous credentials literature. When user privacy is also considered, such solutions typically follow a *blind-then-ring* pattern: the user first obtains a token (potentially in the form of a blind signature) from a single issuer and then wraps it into a ring defined over a designated set of public keys, usually accompanied by a set membership proof to certify the issuer's eligibility. That is, issuer privacy is achieved solely through user-side post-processing, leaving issuers exposed to malicious users during issuance, thereby enabling low-cost, targeted resource exhaustion, as well as issuer-based request correlation and user profiling. In contrast, aiming at an issuance hiding OTAT, we adopt the *blind-and-ring* pattern, where the ring structure is incorporated from the outset of the blind issuance protocol, thereby enforcing ring-based issuer privacy and user privacy jointly. This naturally leads us to use BRS schemes as the core building block. Building atop the solid foundation of canonical blind signatures [29], we modernize the BRS formalization by introducing an explicit three-move structure that accounts for concurrent security. This modification carries over to OTAT: we adopt a three-move, issuer-initiated issuance protocol rather than the two-move, user-initiated protocol found in the literature. The resulting iToken scheme is constructed directly from a BRS scheme, with its security guarantees derived accordingly.

[1] We will open-source iToken upon publication.

We now provide a high-level overview of our BRS construction. Our starting points are the linear function (LF) family-based canonical blind signature scheme [29] and the Type-T^* DualRing framework [57]. Although both primitives share a three-move *commitment-challenge-response* structure, a naïve composition fails to simultaneously ensure unforgeability and blindness while preserving signer-anonymity. Indeed, in [29], the challenge is formed as $c' = c + \beta$, where the pseudo challenge c is derivable from the published signature components and β is the blinding factor. Adapting this mechanism to the ring setting with n members requires the signer to sample $n-1$ "dummy" challenges and compute its own challenge *s.t.* their sum equals c'. Consequently, ensuring blindness by hiding either β or c' without a sufficient binding relation between the two enables forgery, due to the linearity of the summation.

To address this tension, we develop two generic constructions: $\mathsf{BRS}_{\mathsf{HE}}$ from an LF family and a homomorphic encryption (HE) scheme (Section 3.2), and $\mathsf{BRS}_{\mathsf{CP}}$ from an LF and a commit-and-prove (CP) sum argument protocol (Section 3.3). We then provide concrete instantiations of these primitives (in Section 3.4).

In $\mathsf{BRS}_{\mathsf{HE}}$, the user samples a set of per-member blinding factors $\{\beta_j\}_{j\in[n]}$ and binds their sum $\beta = \sum_{j\in[n]} \beta_j$ into the hash that derives c (and hence $c' = c + \beta$, since β is eventually revealed in the signature). It then encrypts, using HE, all elements of $(\{\beta_j\}_{j\in[n]}, c')$ and sends only the resulting ciphertexts to the signer. The signer leverages the homomorphic property of HE to combine ciphertexts and derive the encrypted challenge corresponding to its own index, and subsequently computes the response without decrypting any intermediate values. Notably, the HE-based construction preserves the exact ring signature structure throughout the signing process. This structural transparency facilitates further exploration and refinement of the ring component itself. Leveraging linear identities among artifacts in $\mathsf{BRS}_{\mathsf{HE}}$, our $\mathsf{BRS}_{\mathsf{CP}}$ construction collapses the per-member blinding factors into a single aggregate β. Fixing an index l deterministically derived from the ring, the blinding is encoded by assigning $\beta_l = \beta$, and $\beta_j = 0$ for all $j \neq l$. Since we allow the signer to receive c' in the clear during the signing process, β must not be revealed in the final signature. To prevent message-substitution forgeries that arise when the blinding is left unbound, $\mathsf{BRS}_{\mathsf{CP}}$ commits β to R_β and binds R_β into the hash that derives c. The deterministic placement of the blinding factor breaks the direct per-index alignment between c'_j and pk_j required for verification in DualRing [57] and preserved by $\mathsf{BRS}_{\mathsf{HE}}$. Nevertheless, we observe that the sum argument relation underpinning the efficient, logarithmic-size DualRing variant [57] remains intact. Therefore, we follow the CP paradigm and augment this relation with R_β. The resulting $\mathsf{BRS}_{\mathsf{CP}}$ inherits the same asymptotic efficiency benefits.

Finally, we note that the three-move structure underlying BRS inherits a concurrency-related reduction limitation stemming from the ROS problem [29,23,7,32]. Concretely, the unforgeability guarantee is most meaningful when the number of concurrently opened signing sessions is polylogarithmic in the security parameter. In practice, the ring setting supports higher effective concurrency by obscuring the actual signer within each session, and this can be further secured by requiring signers to manage concurrent signing sessions in a stateful manner. In the literature, there are generic concurrency-boosting transforms [33,15] and blind Schnorr variants designed to avoid the ROS-incurred limitation [52,19]. Building on the latter, we observe that both our HE-based and CP sum argument-based techniques are compatible with such a foundation, which motivates our construction BRS_1 (Section 3.5).

1.3 Related Works

As shown in Table 1, a broad range of OTAT schemes have been proposed following Privacy Pass, including blind signatures [36], equivalence-class signatures [53], and algebraic MACs [17]. These schemes can be categorized based on whether token verification is public or private, depending on whether the issuer and the verifier are distinct parties (public verification [8]) or the same entity (private verification [36]). Another line of work allows metadata to be included in tokens, which may be either public (*e.g.*, revocation information or usage policies [50]) or private (*e.g.*, hidden flags used for abuse detection or access control [36]).

Issuer-Hiding OTATs. To mitigate targeted DoS attacks and improve token unlinkability, a range of OTAT solutions was proposed incrementally, from Privacy-Pass with a private metadata bit (PMB) to an issuer-hiding approach. The PMB-approach embeds a PMB into tokens and enables the issuer to support its fraud-detection mechanisms [36,4]. PMB-hiding enables the verifier to reject marked tokens at verification time, thereby discouraging persistent malicious requests [17]. However, the issuer (authority server) remains exposed during token issuance; as a result, the PMB-hiding approach provides limited protection against targeted DoS attacks on the authorization server during issuance. Issuer-hiding OTATs aim to eliminate linkability to a single root of trust, mitigating risks such as DoS attacks targeting that root. Although practical anonymous credential systems with issuer-hiding capabilities have been deployed [9,46,47,34], only one concrete OTAT construction has so far achieved issuer hiding

Table 1: Comparison with Other Related OTAT Protocols.

Schemes	Moves	PM	PV	IH	Tools
Privacy-Pass [20]	2	✗	✗	✗	Oblivious PRF
PMBT [36]	2	✗	✗	✗	Okamoto–Schnorr
PMBTB [36]	2	✗	✗	✗	Blind Signature
ATPM [51]	2	✓	✗	✗	Blind Signature
ATP&PM [51]	2	✓	✗	✗	Blind Signature
ATPM&PV [51]	2	✓	✓	✗	bilinear pairings
ATHM [17]	2	✗	✗	✗	Algebraic MAC
HinToken-I [53]	2	✗	✗	✗	Equivalence-Class Signature
iToken via $\mathsf{BRS}_{\mathsf{HE}}$	3	✓	✓	✓	LF and HE (modules p)
iToken via $\mathsf{BRS}_{\mathsf{CP}}$	3	✓	✓	✓	LF and CP Sum Argument

* We represent "public metadata" as "PM", "public verifiability" as "PV", "Issuance Hiding" as "IH". "Tools" mean "Building Blocks" for constructions.
* We use a checkmark (*i.e.*, ✓) to indicate that the scheme possesses a particular property, and a cross (*i.e.*, ✗) to denote its absence.

at the verification phase [53]. To date, no OTAT solution provides issuer hiding during issuance, leaving the authorization infrastructure vulnerable to targeted resource exhaustion attacks.

Blind Signatures and Oblivious PRFs. Most existing OTAT constructions are built on (V)OPRFs or blind signatures, and only Karantaidou *et al.* [31] equipped OTAT with a blind multisignature to support decentralized issuance. However, schemes based on blind Schnorr or Okamoto-Schnorr signatures do not naturally support concurrent token issuance. Blind signatures were introduced by Chaum [18] to enable untraceable and unlinkable e-coins. Early schemes relied on factoring, until Camenisch *et al.* [11] proposed the first discrete-log-based blind signature supporting message recovery. Mohammed *et al.* [39] extended this line with a generalized ElGamal-based scheme producing distinct signatures for repeated signings. In contrast, Pointcheval *et al.* [44] provided a provably secure Okamoto-Schnorr-based construction, which inspired our work. Moreover, existing public, verifiable anonymous token schemes primarily focus on protecting user privacy, while the privacy of issuers remains largely unaddressed.

Ring Signatures and Blind Ring Signatures. Ring signatures provide anonymity by enabling a signer to produce a signature on behalf of an arbitrary set of public keys without requiring any group setup or coordination among the members of the set [45]. A wide range of constructions has been proposed to: (i) improve efficiency [57], *e.g.*, by reducing signature size and verification cost; (ii) enhance security [38,24], *e.g.*, through support for linkability, traceability, collusion resistance, and post quantum security [21,56]; and (iii) extend functionality, *e.g.*, via threshold, aggregate, and blind variants. Notably, the DualRing framework [57] constructs two interlocking rings of commitments and challenges, enabling logarithmic-size signatures through an improved Bulletproofs-style argument system. However, blind signatures protect only the signer's privacy, while ring signatures provide anonymity only within a set of public keys; neither alone suffices when both anonymity and untraceability are required. To achieve both simultaneously, a blind ring signature (BRS) is introduced [16] based on blind Schnorr (resp. Okamoto–Schnorr) signatures. Wu *et al.* [55] later proposed a static blind ring signature with constant-size parameters under the extended ROS assumption.

These related works inspire us to design an OTAT scheme that provides strong privacy guarantees for both issuers and users. To this end, we design a new BRS building upon [29,57], which simultaneously achieves signer-anonymity via the ring structure and blindness via blind signing. This combination is essential for constructing OTAT systems that support issuance hiding together with strong user privacy.

1.4 Potential Applications

In addition to mitigating token reuse and targeted DDoS attacks, we investigate expanding iToken to provide a single-use token to indicate authorization. We provide two useful applications and guide our subsequent work in the future:

– Authorization for AI Agent. AI agents typically authenticate via Long-lived API keys, OAuth 2.0 client credentials (machine-to-machine), or User-delegated tokens (*e.g.*, access_token with scopes like `agent:write`). In

particular, an AI agent needs to call backend APIs (*e.g.*, data retrieval and tool use), and OAuth Client Credentials can be replaced with iToken. (i) User consents in the browser, and the frontend requests OTAT from the issuer ring. (ii) Issuer issues OTAT tied to scope, and sends the OTAT to the agent, who then uses it to call APIs. (iii) Verifier accepts OTAT if it is from the issuer ring, scope matches the request, and it has not been previously used.

– Authorization for Atomic Swaps. In standard HTLC-based atomic swaps, participants prove ownership of funds via cryptographic signatures (*e.g.*, ECDSA), and there is no explicit "authorization" layer for who is allowed to initiate or participate in a swap. In an institutional DeFi setting, we need to ensure only eligible parties (*e.g.*, KYC'd users, licensed agents) can swap, prevent Sybil attacks or spam swaps that congest chains, and hide participant identity to avoid front-running or profiling. Thus, iToken enables adding a layer of privacy-preserving, verifiable authorization without sacrificing decentralization or efficiency.

2 Preliminary

Notations. Let $\lambda \in \mathbb{N}$ denote the security parameter. $\mathsf{poly}(\cdot)$ and $\mathsf{negl}(\cdot)$ denote a polynomial and a negligible function, respectively. PPT is short for probabilistic polynomial time. We use ":=" for deterministic assignment, "$\leftarrow$" for assignment to an algorithm's output, and "$\leftarrow\!\!\$$" for uniform random sampling. The integer range $[a \,..\, b]$ represents $\{a, a+1, \ldots, b\}$, $[n]$ denotes $[1 \,..\, n]$, and $[n]_0$ denotes $[0 \,..\, n]$. For a pair of algorithms Alg_1 and Alg_2, $\langle \mathrm{Alg}_1, \mathrm{Alg}_2 \rangle$ denotes a potentially two-party interactive protocol.

(Pseudo) modules. Following the convention of [29], we say that a set R forms a module over a set M if R is a ring with multiplicative identity element 1_R, M is an additive Abelian group, and there exists a mapping [2] $\cdot\colon R \times M \to M$ *s.t.* the following equations hold for all $r, s \in R$ and $x, y \in M$: (1) $r \cdot (x + y) = r \cdot x + r \cdot y$; (2) $(r + s) \cdot x = r \cdot x + s \cdot x$; (3) $(rs) \cdot x = r \cdot (s \cdot x)$; and (4) $1_R \cdot x = x$. Analogously, a set R forms a *pseudo* module over a set M if R and M are additive Abelian groups and there exists a mapping $\cdot\colon R \times M \to M$ *s.t.*: for all $r \in R$ and $x, y \in M$, $r \cdot (x + y) = r \cdot x + r \cdot y$. Moreover, we write $x - r \cdot y$ to denote $x + (-r) \cdot y$ and $x + r \cdot (-y)$ for the mapping applied to $r \in R$ and $-y \in M$.

The following introduces the formal definitions of cryptographic primitives, including a linear function (LF) family, a homomorphic encryption (HE) scheme, a commitment scheme, and a non-interactive argument protocol. Assumptions underlying concrete instantiations are deferred to Section A.1.

2.1 Linear Function Family

Introduced in [3] and later refined by [29], an LF family consists of a tuple of algorithms $\mathsf{LF} = (\mathrm{PGen}, \mathrm{F}, \Psi)$.

– $\mathrm{pp} \leftarrow \mathrm{PGen}(1^\lambda)$: On input the security parameter λ, the parameter generation algorithm outputs public parameters pp, which implicitly define a scalar set $\mathcal{S}$, a domain $\mathcal{D}$, and a range $\mathcal{R}$. Moreover, $\mathcal{D}$ and $\mathcal{R}$ form *pseudo modules* over $\mathcal{S}$ and $|\mathcal{R}| \geq |\mathcal{S}| \geq 2^{2\lambda}$. Since pp is taken by all following algorithms in LF, we omit it when the context is clear.
– $z \leftarrow \mathrm{F}(x)$: On input a point $x \in \mathcal{D}$, the deterministic evaluation function returns $z \in \mathcal{R}$. For any $\mathrm{pp} \leftarrow \mathrm{PGen}(1^\lambda)$, the following properties hold for $\mathrm{F}(\cdot)$.
  - *Pseudo module homomorphism*: For all $x, y \in \mathcal{D}$ and $s \in \mathcal{S}$, $\mathrm{F}(s \cdot x + y) = s \cdot \mathrm{F}(x) + \mathrm{F}(y)$.
  - *Pseudo torsion-free element from the kernel*: There exists $x^* \in \mathcal{D} \setminus \{0\}$ such that (1) $\mathrm{F}(x^*) = 0$, and (2) for all distinct $s, s' \in \mathcal{S}$, $s \cdot x^* \neq s' \cdot x^*$, implying that $\mathrm{F}(\cdot)$ is a many-to-one mapping.
  - *Smoothness*: For any $x \leftarrow\!\!\$\, \mathcal{D}$, $\mathrm{F}(x)$ is uniform over $\mathcal{R}$.
– $x \leftarrow \Psi(y, s, s')$: On input a point $y \in \mathcal{R}$ and scalars $s, s' \in \mathcal{S}$, the deterministic distributor function outputs a point $x \in \mathcal{D}$. For all $\mathrm{pp} \leftarrow \mathrm{PGen}(1^\lambda)$, $x \in \mathcal{D}$, and $s, s' \in \mathcal{S}$, the following holds:

$$(s + s') \cdot \mathrm{F}(x) = s \cdot \mathrm{F}(x) + s' \cdot \mathrm{F}(x) + \mathrm{F}\left(\Psi(\mathrm{F}(x), s, s')\right).$$

As noted in [29], Ψ acts as a *correction term* to treat pseudo modules as if the operation $+$ over $\mathcal{S}$ distributes over $\mathcal{R}$. Specifically, this work restricts to the case where Ψ is identically zero [3], which implies that $\mathcal{D}$ and $\mathcal{R}$ form full-fledged modules with $\mathcal{S}$. For security [4], we recall the definition of collision resistance from [29].

[2] We adopt the same inline fashion as [29], *i.e.*, $r \cdot x$ instead of $\cdot(r, x)$.
[3] The rationale behind this restriction is provided in Remark 2.
[4] Another property considered in *loc. cit.* is ROS (Random inhomogeneities in an Overdetermined, Solvable system of linear equations) hardness, a direct generalization of the ROS problem [49] to LF families, which we use implicitly.

Definition 1 (Collision Resistance). *An LF family satisfies collision resistance if, for any PPT adversary $\mathcal{A}$, the following probability is* $\mathsf{negl}(\lambda)$ *for any* λ *and* $\mathrm{pp} \leftarrow \mathrm{PGen}(1^\lambda)$.

$$\Pr\left[\mathrm{F}(x_1) = \mathrm{F}(x_2) \wedge x_1 \neq x_2 | (x_1, x_2) \leftarrow \mathcal{A}^{\mathrm{H}}(\mathrm{pp})\right]$$

2.2 Homomorphic Encryption Scheme

An HE scheme consists of a tuple of algorithms $\mathsf{HE} = (\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Dec}, \mathrm{Eval})$. We recall the formal syntax from [14].

- $(ek, dk) \leftarrow \mathrm{KeyGen}(1^\lambda)$: On input the security parameter λ, the key generation algorithm outputs an encryption-decryption key pair (ek, dk), which implicitly defines a message space $\mathcal{M}$.
- $\mathrm{ct} \leftarrow \mathrm{Enc}(ek, m)$: On input an encryption key ek and a message $m \in \mathcal{M}$, the encryption algorithm outputs a ciphertext ct. When ek is clear from the context, we may simply write $\mathrm{ct}(m)$ as shorthand for $\mathrm{Enc}(ek, m)$.
- $\mathrm{ct} \leftarrow \mathrm{Eval}(ek, f, \mathrm{ct}_1, \ldots, \mathrm{ct}_t)$: On input ek, an arithmetic circuit $f : \mathcal{M}^t \to \mathcal{M}$ (in a class of "permitted" circuits $\mathcal{F}$), and t ciphertexts, the evaluation algorithm outputs a ciphertext ct.
- $m \leftarrow \mathrm{Dec}(dk, \mathrm{ct})$: On input a decryption key dk and a ciphertext ct, the deterministic decryption algorithm outputs a message m.

This work considers arithmetic circuits for Eval built only from plaintext addition $(+)$, subtraction $(-)$, and scalar multiplication $(\cdot)$, as these operations suffice for our purposes. For simplicity, we lift them to ciphertexts and denote the corresponding homomorphic forms by $(\oplus, \ominus, \odot)$. For example, we write $\mathrm{ct}(m_1) \oplus \mathrm{ct}(m_2)$ as shorthand for $\mathrm{Eval}(ek, +(\cdot,\cdot), \mathrm{ct}(m_1), \mathrm{ct}(m_2))$ and, for a known scalar s, $s \odot \mathrm{ct}(m)$ as shorthand for $\mathrm{Eval}(ek, \cdot(s,\cdot), \mathrm{ct}(m))$.

An HE scheme should satisfy correctness and semantic security. The formal definitions are presented as follows.

Definition 2 (Correctness). *An HE scheme correctly evaluates a family of circuits $\mathcal{F}$ if the following probability is* $1 - \mathsf{negl}(\lambda)$ *for any* λ, $f \in \mathcal{F}$, *and* $\{m_i\}_{i\in[t]} \in \mathcal{M}^t$.

$$\Pr\left[\begin{array}{c}\mathrm{Dec}(dk, \mathrm{Eval}(ek, f, \mathrm{ct}_1, \ldots, \\ \mathrm{ct}_t)) = f(m_1, \ldots, m_t)\end{array} \middle| \begin{array}{c}(ek, dk) \leftarrow \mathrm{KeyGen}(1^\lambda) \\ \mathrm{ct}_i \leftarrow \mathrm{Enc}(ek, m_i), \forall i \in [t]\end{array}\right]$$

Definition 3 (Semantic Security). *An HE scheme satisfies semantic security if, for any PPT adversary $\mathcal{A}$, the following probability is* $\mathsf{negl}(\lambda)$ *for any* λ.

$$\left|\Pr\left[b' = b \middle| \begin{array}{c}(ek, dk) \leftarrow \mathrm{KeyGen}(1^\lambda) \\ (m_0, m_1) \leftarrow \mathcal{A}(ek); b \leftarrow_{\$} \{0,1\} \\ \mathrm{ct}_b \leftarrow \mathrm{Enc}(ek, m_b); b' \leftarrow \mathcal{A}(\mathrm{ct}_b)\end{array}\right] - \frac{1}{2}\right|$$

2.3 Commitment Scheme

We recall the syntax of a (non-interactive) commitment scheme [5] $\mathsf{COM} = (\mathrm{Setup}, \mathrm{Commit}, \mathrm{Verify})$ and its properties of correctness, binding, and hiding.

- $ck \leftarrow \mathrm{Setup}(1^\lambda)$: On input the security parameter λ, the setup algorithm outputs a public commitment key ck, which includes the descriptions of an input space $\mathcal{M}$, a commitment space $\mathcal{C}$, and an opening space $\mathcal{O}$.
- $(R, O) \leftarrow \mathrm{Commit}(ck, m)$: On input a commitment key ck and a value $m \in \mathcal{M}$, the commitment algorithm outputs a commitment $R \in \mathcal{C}$ and an opening $O \in \mathcal{O}$.
- $b \leftarrow \mathrm{Verify}(ck, R, m, O)$: The deterministic verification algorithm outputs a bit: $b = 1$ if the input tuple is valid, or $b = 0$ otherwise.

Definition 4 (Correctness). *A commitment scheme satisfies correctness if the following probability holds for any* λ *and* $m \in \mathcal{M}$.

$$\Pr\left[\mathrm{Verify}(ck, R, m, O) = 1 \middle| \begin{array}{c}ck \leftarrow \mathrm{Setup}(1^\lambda) \\ R \leftarrow \mathrm{Commit}(ck, m)\end{array}\right] = 1$$

[5] There exists more than one syntax for commitment schemes in the literature. We adopt the one from the commit-and-prove paradigm proposal [5], which aligns with our intended usage.

Definition 5 (Binding). *A commitment scheme is (computational) binding if, for any PPT adversary $\mathcal{A}$, the following probability is* $\mathsf{negl}(\lambda)$ *for any* λ.

$$\Pr\left[\begin{array}{c}\mathrm{Verify}(ck,R,m,O)=1\wedge\\ \mathrm{Verify}(ck,R,m',O')=1\wedge\\ m\neq m'\end{array}\;\middle|\;\begin{array}{c}ck\leftarrow\mathrm{Setup}(1^\lambda)\\ (R,m,O,m',O')\leftarrow\mathcal{A}(ck)\end{array}\right]$$

Definition 6 (Hiding). *A commitment scheme is (perfect) hiding if the following distribution ensembles are identical, i.e., statistical distance is* 0*, for any* λ*,* $ck\leftarrow\mathrm{Setup}(1^\lambda)$*, and any* $m_0,m_1\in\mathcal{M}$.

$$\mathcal{D}_0:=\{\mathrm{Commit}(ck,m_0)\}\text{ and }\mathcal{D}_1:=\{\mathrm{Commit}(ck,m_1)\}$$

2.4 Non-interactive Argument of Knowledge

Let $\mathcal{R}\subseteq\{0,1\}^*\times\{0,1\}^*$ be an NP relation and H be a random oracle, both parameterized by λ. Denote the language defined by $\mathcal{R}$ as $L_\mathcal{R}:=\{x\in\{0,1\}^*|\exists w\in\{0,1\}^{\mathsf{poly}(|x|)}\ s.t.\ (x,w)\in\mathcal{R}\}$. A non-interactive argument protocol for $\mathcal{R}$ consists of a tuple of algorithms $\Pi_\mathcal{R}=(\mathrm{Setup},\mathrm{Prove}^\mathrm{H},\mathrm{Verify}^\mathrm{H})$.

- $(\mathrm{crs},\tau)\leftarrow\mathrm{Setup}(1^\lambda,\mathcal{R})$: On input the security parameter λ, Setup outputs a common reference string crs and a trapdoor τ.
- $\pi\leftarrow\mathrm{Prove}^\mathrm{H}(\mathrm{crs},x,w)$: On input a crs, an instance x, and a witness w, the prove algorithm outputs an argument π.
- $b\leftarrow\mathrm{Verify}^\mathrm{H}(\mathrm{crs},x,\pi)$: The deterministic verification algorithm outputs a bit $b\in\{0,1\}$: $b=1$ if π is valid proof for the instance x, or $b=0$ otherwise.

We recall the notions of completeness, zero-knowledge, and knowledge soundness [22], which yield a non-interactive zero-knowledge argument of knowledge (NIZKAoK).

Definition 7 (Completeness). *A non-interactive argument protocol for a relation $\mathcal{R}$ satisfies completeness if the following probability holds for any λ, $x\in L_\mathcal{R}$, and w s.t. $(x,w)\in\mathcal{R}$.*

$$\Pr\left[\mathrm{Verify}^\mathrm{H}(\mathrm{crs},x,\pi)=1\;\middle|\;\begin{array}{c}(\mathrm{crs},\tau)\leftarrow\mathrm{Setup}(1^\lambda,\mathcal{R})\\ \pi\leftarrow\mathrm{Prove}^\mathrm{H}(\mathrm{crs},x,w)\end{array}\right]=1$$

Definition 8 (Zero-Knowledge). *A non-interactive argument for $\mathcal{R}$ is (statistical) zero-knowledge if, for any PPT adversary $\mathcal{A}$, there exists a PPT simulator* Sim^H *s.t. the following distribution ensembles are statistically close for any λ, $(x,w)\in\mathcal{R}$, and* $(\mathrm{crs},\tau)\leftarrow\mathrm{Setup}(1^\lambda,\mathcal{R})$.

$$\mathcal{D}_0:=\{\mathrm{Prove}^\mathrm{H}(\mathrm{crs},x,w)\}\text{ and }\mathcal{D}_1:=\{\mathrm{Sim}^\mathrm{H}(\mathrm{crs},\tau,x)\}$$

Definition 9 (Knowledge Soundness). *A non-interactive argument for $\mathcal{R}$ is an argument of knowledge if, for any* PPT *adversary $\mathcal{A}$, there exists a PPT extractor* $\mathrm{E}^\mathcal{A}$ *(with black-box access to $\mathcal{A}$) s.t. the following probability is* $\mathsf{negl}(\lambda)$ *for any λ.*

$$\Pr\left[\begin{array}{c}\mathrm{Verify}^\mathrm{H}(\mathrm{crs},x,\pi)=1\wedge\\ (x,w)\notin\mathcal{R}\end{array}\;\middle|\;\begin{array}{c}(\mathrm{crs},\tau)\leftarrow\mathrm{Setup}(1^\lambda,\mathcal{R})\\ (x,\pi)\leftarrow\mathcal{A}^\mathrm{H}(\mathrm{crs})\\ w\leftarrow\mathrm{E}^\mathcal{A}(\mathrm{crs})\end{array}\right]$$

3 The Core Building Block: Canonical Blind Ring Signatures

This section presents the core building block of our iToken: a blind ring signature (BRS) scheme. We provide an up-to-date BRS formalization that departs from the convention [26] by modeling the signing process explicitly as a three-move protocol, which we refer to as a *canonical three-move BRS*. This modification aligns with the recent advances in blind and ring signature literature [29,23,57] and offers greater flexibility in formulating security notions under concurrent executions. We then present two generic, provably secure constructions, together with concrete, efficient instantiations. Finally, we note that our constructions inherit a well-known caveat of three-move blind signatures, namely susceptibility to ROS-style attacks [29,7,32]; we discuss its impact in the ring setting, outline mitigation strategies, and give a concrete BRS construction based on a ROS-resistant blind Schnorr variant [52] in Section 3.5.

3.1 BRS Formalization Revisited

A canonical three-move BRS scheme consists of a tuple of algorithms $\mathsf{BRS} = (\text{Setup}, \text{KeyGen}, \langle \text{Sign}, \text{User}\rangle, \text{Verify})$, where the interactive signing protocol is further represented by $\langle \text{Sign}, \text{User}\rangle = (\text{Sign}_1, \text{User}_1, \text{Sign}_2, \text{User}_2)$.

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: On input the security parameter λ, the setup algorithm outputs the public parameter pp, which defines a message space $\mathcal{M}$. Since pp is taken by all following algorithms in BRS, we omit it when the context is clear.
- $(sk_i, pk_i) \leftarrow \text{KeyGen}(i)$: On input a signer index $i \in \mathbb{N}$, the key generation algorithm outputs a secret-public key pair (sk_i, pk_i).
- $\langle \text{Sign}(sk_i, \mathbf{pk}), \text{User}(\mathbf{pk}, m)\rangle$ is an interactive protocol between a signer i in a ring $\mathbf{pk}$ with a secret key sk_i (*s.t.* the corresponding $pk_i \in \mathbf{pk}$) and a user holding a message $m \in \mathcal{M}$. The three concrete moves are detailed as follows.
  - $(R, \mathsf{state}_\text{S}) \leftarrow \text{Sign}_1(sk_i, \mathbf{pk})$: Run by the signer, the algorithm outputs a commitment R and the signer's state state_S.
  - $(c, \mathsf{state}_\text{U}) \leftarrow \text{User}_1(\mathbf{pk}, R, m)$: Run by the user, the algorithm outputs a challenge c and the user's state state_U.
  - $z \leftarrow \text{Sign}_2(\mathsf{state}_\text{S}, c)$: On input the signer's state_S and a challenge c, the algorithm outputs a response z deterministically.
  - $\sigma \leftarrow \text{User}_2(\mathsf{state}_\text{U}, z)$: On input the user's state_U and a response z, the algorithm outputs a signature σ deterministically, where, possibly, $\sigma = \perp$.

  For simplicity, we denote the output of the protocol as (b, σ), where $b = 0$ if any algorithm aborts or $\sigma = \perp$; otherwise, $b = 1$.
- $b \leftarrow \text{Verify}(\mathbf{pk}, m, \sigma)$: The deterministic verification algorithm outputs a bit $b \in \{0, 1\}$: $b = 1$ if σ is a valid signature on the message m *w.r.t.* the ring $\mathbf{pk}$, or $b = 0$ otherwise.

A BRS scheme should satisfy correctness, unforgeability, signer-anonymity, and blindness. These notions are inherited from the blind and ring signature literature and carry the same high-level intuition. *Unforgeability* ensures that no adversary can produce more valid signatures (*w.r.t.* an uncorrupted ring) than the number of signing sessions it successfully completes. *Signer-anonymity* guarantees that no adversary can determine which ring member it is interacting with in a given signing session. Finally, *blindness* requires that no adversary can learn the message being signed nor link a valid signature tuple to a specific signing session. Formally:

Definition 10 (Correctness). *A BRS scheme is correct if the following probability equals* 1 *for any* λ *and* $\text{pp} \leftarrow \text{Setup}(1^\lambda)$*, any signer index* $i \in [n]$*, and message* $m \in \mathcal{M}$*.*

$$\Pr\left[\begin{array}{c} b = 1 \wedge \\ \text{Verify}(\mathbf{pk}, m, \sigma) = 1 \end{array} \middle| \begin{array}{c} (sk_i, pk_i) \leftarrow \text{KeyGen}(i), \forall i \in [n] \\ (b, \sigma) \leftarrow \langle \text{Sign}(sk_i, \mathbf{pk}), \text{User}(\mathbf{pk}, m)\rangle \end{array}\right],$$

where $\mathbf{pk} = \{pk_i\}_{i \in [n]}$.

To capture the security properties, we begin by defining the oracles, adapting those from recent blind signatures [29,23] to the ring setting. Unlike legacy formulations that model signing and obtaining via a single monolithic oracle [41,25,27], our three-move structure allows us to split the oracles and explicitly track sessions.

- $\mathcal{O}^{C}_{\text{Corrupt}}(i)$ is a corruption oracle that maintains a set of corrupted public keys C, initialized as $C := \emptyset$. On queried a signer index i, the oracle returns sk_i and records the corresponding pk_i in C, *i.e.*, $C = C \cup \{pk_i\}$.
- $(\mathcal{O}_{\text{Sign}_1}, \mathcal{O}_{\text{Sign}_2})$ are oracles that imitates the behavior of a signer.
  - $\mathcal{O}^{sid, S}_{\text{Sign}_1}(i, \mathbf{pk})$ maintains a session identifier $sid \in \mathbb{N}_0$ and a set of opened sessions S, which are initialized as $sid := 0$ and $S := \emptyset$, respectively. On queried a signer index i and a ring $\mathbf{pk}$, the oracle aborts with $\perp$ if $pk_i \notin \mathbf{pk}$. Otherwise, it performs $sid := sid + 1$, $(R, \mathsf{state}_{sid}) \leftarrow \text{Sign}_1(sk_i, \mathbf{pk})$, $S := S \cup \{sid\}$, and returns (sid, R). Moreover, state_{sid} is retained for $\mathcal{O}_{\text{Sign}_2}$.
  - $\mathcal{O}^{S, q}_{\text{Sign}_2}(sid, c)$ inherits the set of opened sessions S and the corresponding states from $\mathcal{O}_{\text{Sign}_1}$. It also maintains a counter $q \in \mathbb{N}_0$ for the number of finished sessions, initialized as $q := 0$. On queried a session identifier sid and a challenge c, the oracle aborts with $\perp$ if $sid \notin S$. Otherwise, it performs $z \leftarrow \text{Sign}_2(\mathsf{state}_{sid}, c)$, $S := S \setminus \{sid\}$, $q := q + 1$, and returns z.
- $(\mathcal{O}_{\text{SignLoR}_1}, \mathcal{O}_{\text{SignLoR}_2})$ is a pair of oracles that facilitates the challenge signing session *w.r.t.* a challenge bit $b \in \{0, 1\}$ for defining signer-anonymity.

- $\mathcal{O}^{sess,b}_{\text{SignLoR}_1}(i_0, i_1, \mathbf{pk})$. On queried two signer indices i_0, i_1 and a ring $\mathbf{pk}$, the oracle aborts with $\perp$ if $pk_{i_0} \notin \mathbf{pk} \vee pk_{i_1} \notin \mathbf{pk}$. Otherwise, it opens a session with $sess := \mathsf{open}$, performs $(R_b, \mathsf{state}_{sid}) \leftarrow \text{Sign}_1(sk_{i_b}, \mathbf{pk})$, and returns R_b. Moreover, state_{sid} is retained for $\mathcal{O}_{\text{SignLoR}_2}$.
- $\mathcal{O}^{sess,b}_{\text{SignLoR}_2}(c)$ inherits the state_{sid} from $\mathcal{O}_{\text{SignLoR}_1}$. On queried a challenge c, the oracle aborts with $\perp$ if $sess \neq \mathsf{open}$. Otherwise, it performs $z \leftarrow \text{Sign}_2(\mathsf{state}_{sid}, c)$ and returns z.

– $(\mathcal{O}_{\text{Init}}, \mathcal{O}_{\text{User}_1}, \mathcal{O}_{\text{User}_2})$ is a triple of oracles that simulates a user performing two signing sessions, denoted by $(sess_0, sess_1)$, with the adversary impersonating a malicious signer. It serves as the challenge oracle for defining blindness *w.r.t.* a challenge bit $b \in \{0, 1\}$. Moreover, $b_0 := b$ and $b_1 := 1 - b$.
  - $\mathcal{O}^{sess_0,sess_1,b}_{\text{Init}}(\mathbf{pk}, m_0, m_1)$. On queried a ring $\mathbf{pk}$ and two messages $m_0, m_1 \in \mathcal{M}$ chosen by the adversary, the oracle initializes both session states as: $sess_0 := \mathsf{init}, sess_1 := \mathsf{init}$.
  - $\mathcal{O}^{sess_0,sess_1,b}_{\text{User}_1}(sid, R_{sid})$. On queried a session identifier sid and a commitment R_{sid}, the oracle aborts with $\perp$ if $sid \notin \{0, 1\} \vee sess_{sid} \neq \mathsf{init}$. Otherwise, it sets the session state to open $sess_{sid} := \mathsf{open}$, performs $(c_{sid}, \mathsf{state}_{sid}) \leftarrow \text{User}_1(\mathbf{pk}, R_{sid}, m_{b_{sid}})$, and returns c_{sid}.
  - $\mathcal{O}^{sess_0,sess_1,b}_{\text{User}_2}(sid, z_{sid})$. On queried a session identifier sid and a response z_{sid}, the oracle aborts with $\perp$ if $sess_{sid} \neq \mathsf{open}$. Otherwise, it closes the session $sess_{sid} := \mathsf{closed}$ and performs $\sigma_{b_{sid}} \leftarrow \text{User}_2(\mathsf{state}_{sid}, z_{sid})$. Until both sessions closed, *i.e.*, $sess_0 = sess_1 = \mathsf{closed}$, the oracle returns $(\perp, \perp)$ if $\sigma_0 = \perp \vee \sigma_1 = \perp$; otherwise, it returns (σ_0, σ_1).

It suffices to maintain a *single* session, *w.r.t.* the challenge bit b in $(\mathcal{O}_{\text{SignLoR}_1}, \mathcal{O}_{\text{SignLoR}_2})$, for the signer-anonymity experiment, as the adversary's goal is to distinguish the actual signer from the ring within a specific session. This contrasts with blindness, which necessitates two parallel sessions, maintained via $(\mathcal{O}_{\text{Init}}, \mathcal{O}_{\text{User}_1}, \mathcal{O}_{\text{User}_2})$, to define the swap scenario.

In the following definitions, we assume *w.l.o.g.* that the size of the adversary's output ring $\mathbf{pk}^*$ is n. For unforgeability, we adopt the standard $(k, k+1)$-notion [30] and incorporate the insider corruption model [6].

Definition 11 (Unforgeability *w.r.t.* Insider Corruption). *A BRS scheme satisfies $(k, k+1)$-unforgeability w.r.t. insider corruption if, for any PPT adversary $\mathcal{A}$ that has access to $\mathcal{O} := (\mathcal{O}^{C}_{\text{Corrupt}}, \mathcal{O}^{sid,S}_{\text{Sign}_1}, \mathcal{O}^{S,q}_{\text{Sign}_2})$, the following probability is $\mathsf{negl}(\lambda)$ for any λ, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and any $N = \mathsf{poly}(\lambda)$.*

$$\Pr\left[\begin{array}{l|c} \text{Verify}(\mathbf{pk}^*, m_j^*, \sigma_j^*)=1 \wedge & (sk_i, pk_i) \leftarrow \text{KeyGen}(i), \forall i \in [N] \\ m_j^* \neq m_{j'}^*, \forall j, j' \in [k+1] & (\mathbf{pk}^*, \{(m_j^*, \sigma_j^*)\}_{j \in [k+1]}) \\ \wedge \mathbf{pk}^* \subseteq \mathbf{pk} \setminus C \wedge q \leq k & \leftarrow \mathcal{A}^{\mathcal{O}}(\mathbf{pk}) \end{array}\right],$$

where $\mathbf{pk} = \{pk_i\}_{i \in [N]}$. The strong variant requires all (m_j^*, σ_j^*) to be distinct (instead of m_j^*).

As for signer-anonymity, we adopt the strong model (*i.e.*, full key exposure) of [6], where the adversary is given the randomness used in key generation.

Definition 12 (Signer-Anonymity against Full Key Exposure). *A BRS scheme satisfies signer-anonymity against full key exposure if, for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ that has access to $\mathcal{O}_{\text{Sign}} := (\mathcal{O}^{sid,S}_{\text{Sign}_1}, \mathcal{O}^{S,q}_{\text{Sign}_2})$ and $\mathcal{O}_{\text{SignLoR}} := (\mathcal{O}^{sess,b}_{\text{SignLoR}_1}, \mathcal{O}^{sess,b}_{\text{SignLoR}_2})$, respectively, the following probability is $\mathsf{negl}(\lambda)$ for any λ, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and any $N = \mathsf{poly}(\lambda)$.*

$$\left|\Pr\left[\begin{array}{l|c} b' = b \wedge & (sk_i, pk_i) := \text{KeyGen}(i; r_i), \forall i \in [N] \\ pk_{i_0}, pk_{i_1} \in & (i_0, i_1, \mathbf{pk}^*, \mathsf{state}) \leftarrow \mathcal{A}_1^{\mathcal{O}_{\text{Sign}}}(\mathbf{pk}) \\ \mathbf{pk} \cap \mathbf{pk}^* & b \leftarrow_{\$} \{0, 1\} \\ & b' \leftarrow \mathcal{A}_2^{\mathcal{O}_{\text{SignLoR}}}(\mathsf{state}, \{r_i\}_{i \in [N]}) \end{array}\right] - \frac{1}{2}\right|,$$

where $\mathbf{pk} = \{pk_i\}_{i \in [N]}$, and $\mathcal{A}_2$ interacts with the challenge oracles sequentially: it first queries $\mathcal{O}^{sess,b}_{\text{SignLoR}_1}(i_0, i_1, \mathbf{pk}^*, m)$ to obtain R_b, and then adaptively chooses c to query $\mathcal{O}^{sess,b}_{\text{SignLoR}_2}(c)$.

Finally, our blindness follows the *honest signer model* [30] (see Remark 1), adapted to the ring setting: the adversary is given the ring and corresponding secret keys from the experiment. Moreover, as noted by [55,26], the two challenge signatures must be generated *w.r.t.* the same ring; otherwise, blindness is trivially compromised since the ring is public. This constraint is only imposed for the formal treatment and does not preclude practical deployments from using different rings across services.

Definition 13 (Blindness). *A BRS scheme satisfies (statistical) blindness if, for any PPT adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *that has access to* $\mathcal{O} := (\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_1}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_2}^{sess_0,sess_1,b})$*, the following probability is* $\mathsf{negl}(\lambda)$ *for any* λ *and* $\text{pp} \leftarrow \text{Setup}(1^\lambda)$.

$$\left| \Pr\left[b' = b \;\middle|\; \begin{array}{c} \{(sk_i, pk_i) \leftarrow \text{KeyGen}(i)\}_{i\in[n]} \\ (m_0, m_1, \mathsf{state}) \leftarrow \mathcal{A}_1(\{(sk_i, pk_i)\}_{i\in[n]}) \\ b \leftarrow_\$ \{0,1\}; b' \leftarrow \mathcal{A}_2^{\mathcal{O}}(\mathsf{state}) \end{array} \right] - \frac{1}{2} \right|,$$

where the input to $\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}$ *is* $(\mathbf{pk} := \{pk_i\}_{i\in[n]}, m_0, m_1)$.

Note that the ring effectively serves as mutually agreed-upon public information, resembling that in a *partially* blind signature [1]. Leveraging this analogy, we can extend the input ring argument to bind arbitrary data, hence realizing the public metadata functionality of OTAT schemes; we reflect this extension in Section 4.

Remark 1 (On the honest signer model). We stress that "honest signer" refers only to how key pairs are generated, namely via KeyGen, and *not* to the signer's behavior, which is controlled by the malicious adversary in the blindness game. Along the same axis, one can define semi-honest and malicious signer models [54, Definition 3.6] (adapted to BRS in Definitions 16 and 17, respectively): the former lets the adversary choose the randomness used in KeyGen, while the latter lets it fully control key generation (*i.e.* the keys can be malformed). Our statistical blindness proof (and that of [29]) relies only on key well-formedness, and hence extends to the semi-honest signer model. Moreover, per [54, Section 3.6], an NIZKAoK-based black-box transformation upgrades semi-honest signer blindness to malicious signer blindness. Thus, although we present the definition in the honest signer model, our BRS constructions (and hence iToken) can achieve stronger blindness guarantees via known techniques.

3.2 Generic Construction from LF and HE

At a high level, both our generic constructions are built atop the LF-based blind signature framework [29] and the Type-T^* (*i.e.*, Fiat-Shamir over a three-move identification scheme) DualRing framework [57]. Note that the Type-T^* structure is also compatible with an LF representation[6]. We thereby combine the two using an LF-based backbone. The remaining task is to blind the signer's commitment and the user's challenges *w.r.t.* the ring, while ensuring that the signer can recover the challenge for its own index without compromising blindness. Let $\mathsf{LF} = (\text{PGen}, \text{F}, \Psi)$ be an LF family that satisfies $\Psi \equiv 0$ (detailed in Remark 2), and let $\text{H} : \{0,1\}^* \to \mathcal{S}$ be a hash function, where $\mathcal{S}$ denotes the scalar set associated with LF. The first construction $\mathsf{BRS}_{\mathsf{HE}}$, detailed in Figure 2, achieves the aforementioned requirement by additionally employing an HE scheme $\mathsf{HE} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$.

We now explain the rationale behind the signing process. The signer with index $i \in [n]$ first prepares its commitment as $R := \mathsf{LF}.\text{F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j$ (Line 7) for randomly sampled $r \leftarrow_\$ \mathcal{D}$, and $c_j \leftarrow_\$ \mathcal{S}$ for all $j \in [n] \wedge j \neq i$. This computation is identical to that of DualRing [57].

Recall that, in canonical blind signatures [29], the commitment is blinded by adding a linear combination of the LF evaluation at a random blinding point $\alpha \leftarrow_\$ \mathcal{D}$ and the single blinded public key under $\beta \leftarrow_\$ \mathcal{S}$. We extend this step to the ring setting by sampling a blinding factor $\beta_j \leftarrow_\$ \mathcal{S}$ independently for each public keys $pk_j \in \mathbf{pk}, j \in [n]$. Thereby, the user computes the blinded commitment R' as $R' := R + \mathsf{LF}.\text{F}(\alpha) + \sum_{j\in[n]} \beta_j \cdot pk_j$ (Line 11). Next, R' and, departing from existing literature, the sum $\beta := \sum_{j\in[n]} \beta_j$ are bound into a "pseudo" challenge c via the hash function H, *i.e.*, $c := \text{H}(\mathbf{pk}, R', m, \beta)$ (Line 13). Finally, the user sets the (blinded) challenge to $c' := c + \beta$. Since β is taken as input to H when computing c, it must be revealed in the final signature for verification. Accordingly, passing c' directly to the signer *in the signing process*, as in [29], would break blindness: after learning the signature, the signer can reconstruct $\hat{R}'$ from $\{c_j\}_{j\in[n]}$, and hence $\hat{c}$, and then check whether $c' = \hat{c} + \beta$. We circumvent this issue by instead having the user send encryptions of $\{\beta_j\}_{j\in[n]}$ and c' under HE, *i.e.*, $\big(\{\text{ct}(\beta_j)\}_{j\in[n]}, \text{ct}(c')\big)$ (Line 15, 16). The homomorphism allows the signer to derive $\text{ct}(c'_j) := \text{ct}(c_j) \oplus \text{ct}(\beta_j)$ for each $j \in [n] \wedge j \neq i$, where $\text{ct}(c_j)$ is computed locally from c_j. The signer then computes the blinded challenge for index i (as in [29], but in encrypted form) as $\text{ct}(c'_i) := \text{ct}(c') \ominus \bigoplus_{j\in[n]\wedge j\neq i} \text{ct}(c'_j)$ (Line 22), and finally cancels the encrypted blinding factor $\text{ct}(\beta_i)$ by computing $\text{ct}(c'_i) \ominus \text{ct}(\beta_i)$. Its response is set to $\text{ct}(z) := \text{ct}(r) \ominus (\text{ct}(c'_i) \ominus \text{ct}(\beta_i)) \odot sk_i$ (Line 23, via a scalar multiplication of HE). $\{\text{ct}(c'_j)\}_{j\in[n]}$ is additionally sent to the user for final signature production. Note that, by

[6] For completeness, we recall the blind signature construction of [29] and provide an LF-based representation for the DualRing framework of [57] (Section A.2 and Section A.3).

interpreting $c_i := c'_i - \beta_i$, we obtain the following identity:

$$c' = c'_i + \sum_{j\in[n]\wedge j\neq i} c'_j = (c_i + \beta_i) + \sum_{j\in[n]\wedge j\neq i} (c_j + \beta_j)$$
$$= \sum_{j\in[n]} c_j + \sum_{j\in[n]} \beta_j = c + \beta. \tag{3.1}$$

$c_i = c - \sum_{j\in[n]\wedge j\neq i} c_j$ resembles the (unblinded) challenge in [57].

The remaining computation on the user side is rather simple: it decrypts the ciphertexts to obtain $(\{c'_j\}_{j\in[n]}, z)$, checks the sum relation $c' = \sum_{j\in[n]} c'_j$, blinds the signer's response with $z' := z + \alpha$, and outputs the final signature as $\sigma := (\{c'_j\}_{j\in[n]}, z', \beta)$.

Remark 2 (On the restriction of $\Psi \equiv 0$). Consider the signer's response in plaintext $z = r - (c'_i - \beta_i)\cdot sk_i$. Per the LF definition, the evaluation $\mathsf{LF}.\mathrm{F}$ is not strictly linear *w.r.t.* scalar multiplication if its domain $\mathcal{D}$ and range $\mathcal{R}$ form only *pseudo modules* with the scalar set $\mathcal{S}$. In particular, applying $\mathsf{LF}.\mathrm{F}$ to $(c'_i - \beta_i)\cdot sk_i$ introduces a correction term via the distributor function $\mathsf{LF}.\Psi$. Given $\mathsf{LF}.\mathrm{F}(sk_i) = pk_i$, $\mathsf{LF}.\mathrm{F}((c'_i - \beta_i)\cdot sk_i) = (c'_i - \beta_i)\cdot pk_i + \mathsf{LF}.\Psi(pk_i, c'_i, \beta_i)$. However, the user should not learn the signer index i; hence, it cannot identify the public key pk_i to derive the appropriate correction term. One mitigation is to include a correction term for each pk_j (for all $j \in [n]$), which nearly doubles both the computation cost of User_2 and the size of the final signature. For efficiency, we restrict the LF family to $\Psi \equiv 0$, *i.e.*, $\mathcal{D}$ and $\mathcal{R}$ form full-fledged *modules* with $\mathcal{S}$. This restriction still permits concrete instantiations presented in Section 3.4.

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{Setup}(1^\lambda)}$:
00 $\mathsf{LF}.\mathrm{pp} \leftarrow \mathsf{LF}.\mathrm{PGen}(1^\lambda)$
01 Let $\mathrm{H} : \{0,1\}^* \to \mathcal{S}$ be a hash function
02 **return** $\mathrm{pp} := (\mathsf{LF}.\mathrm{pp}, \mathrm{H})$
// Parse $(\lambda, \mathcal{S}, \mathcal{D}, \mathcal{R}, \mathrm{H}) \in \mathrm{pp}$

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{KeyGen}(\mathrm{pp})}$:
03 $sk \leftarrow_\$ \mathcal{D}; pk \leftarrow \mathsf{LF}.\mathrm{F}(sk)$
04 **return** (sk, pk)

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{Sign}_1(sk_i, \mathbf{pk})}$:
05 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$
06 $r \leftarrow_\$ \mathcal{D}; c_j \leftarrow_\$ \mathcal{S}, \forall j \in [n] \wedge j \neq i$
07 $R := \mathsf{LF}.\mathrm{F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j$
08 **return** $(R, \mathsf{state}_\mathrm{S})$

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{User}_1(\mathbf{pk}, R, m)}$:
09 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$
10 $\alpha \leftarrow_\$ \mathcal{D}; \beta_j \leftarrow_\$ \mathcal{S}, \forall j \in [n]$
11 $R' := R + \mathsf{LF}.\mathrm{F}(\alpha) + \sum_{j\in[n]} \beta_j \cdot pk_j$
12 $\beta := \sum_{j\in[n]} \beta_j$
13 $c := \mathrm{H}(\mathbf{pk}, R', m, \beta); c' := c + \beta$
14 $(ek, dk) \leftarrow \mathsf{HE}.\mathrm{KeyGen}(1^\lambda)$
15 $\mathrm{ct}(\beta_j) \leftarrow \mathsf{HE}.\mathrm{Enc}(ek, \beta_j), \forall j \in [n]$
16 $\mathrm{ct}(c') \leftarrow \mathsf{HE}.\mathrm{Enc}(ek, c')$
17 **return** $\left((ek, \{\mathrm{ct}(\beta_j)\}_{j\in[n]}, \mathrm{ct}(c')), \mathsf{state}_\mathrm{U}\right)$

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{Sign}_2(\mathsf{state}_\mathrm{S}, (ek, \{\mathrm{ct}(\beta_j)\}_{j\in[n]}, \mathrm{ct}(c')))}$:
18 Parse $(sk_i, r, \{c_j\}_{j\in[n]\wedge j\neq i})$ from $\mathsf{state}_\mathrm{S}$
19 For all $j \in [n] \wedge j \neq i$:
20 $\quad \mathrm{ct}(c_j) \leftarrow \mathsf{HE}.\mathrm{Enc}(ek, c_j)$
21 $\quad \mathrm{ct}(c'_j) := \mathrm{ct}(c_j) \oplus \mathrm{ct}(\beta_j)$
22 $\mathrm{ct}(c'_i) := \mathrm{ct}(c') \ominus \bigoplus_{j\in[n]\wedge j\neq i} \mathrm{ct}(c'_j)$
23 $\mathrm{ct}(z) := \mathrm{ct}(r) \ominus (\mathrm{ct}(c'_i) \ominus \mathrm{ct}(\beta_i)) \odot sk_i$
24 **return** $(\{\mathrm{ct}(c'_j)\}_{j\in[n]}, \mathrm{ct}(z))$

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{User}_2(\mathsf{state}_\mathrm{U}, (\{\mathrm{ct}(c'_j)\}_{j\in[n]}, \mathrm{ct}(z)))}$:
25 Parse (α, β, c', dk) from $\mathsf{state}_\mathrm{U}$
26 $c'_j := \mathsf{HE}.\mathrm{Dec}(dk, \mathrm{ct}(c'_j)), \forall j \in [n]$
27 **return** $\perp$ if $c' \neq \sum_{j\in[n]} c'_j$
28 $z := \mathsf{HE}.\mathrm{Dec}(dk, \mathrm{ct}(z)); z' := z + \alpha$
29 **return** $\sigma := (\{c'_j\}_{j\in[n]}, z', \beta)$

$\underline{\mathsf{BRS}_{\mathsf{HE}}.\mathrm{Verify}(\mathbf{pk}, m, \sigma)}$:
30 Parse $\sigma = (\{c'_j\}_{j\in[n]}, z', \beta)$
31 $R' := \mathsf{LF}.\mathrm{F}(z') + \sum_{j\in[n]} c'_j \cdot pk_j$
32 $c := \mathrm{H}(\mathbf{pk}, R', m, \beta); c' := c + \beta$
33 **return** 0 if $c' \neq \sum_{j\in[n]} c'_j$
34 **return** 1

Fig. 2: $\mathsf{BRS}_{\mathsf{HE}}$: A Generic BRS from LF and HE

Security analysis. We formally state the security guarantee, and the proof is deferred to Section B.1.

Theorem 1 (Security of $\mathsf{BRS}_{\mathsf{HE}}$). *Let* LF *be an LF family,* H *be a hash function modeled as a random oracle, and* HE *be an HE scheme. The construction* $\mathsf{BRS}_{\mathsf{HE}}$ *in Figure 2 satisfies the following properties in the random oracle model: correctness;* $(k, k+1)$*-unforgeability w.r.t. insider corruption if* LF *is collision resistant; signer-anonymity against full key exposure; and* blindness *if* HE *satisfies semantic security.*

3.3 Generic Construction from LF and Commit-and-Prove Sum Argument

Blinding all public keys in the ring constitutes a natural extension of [29], wherein the user prepares blinded challenges for all ring members, including the signer at index i and the "dummy" ones. Under this structure, an HE layer enforces signer-anonymity and blindness by encrypting both the blinding factors and the blinded challenge. While functionally sound, this approach incurs a scalability bottleneck: communication and computation overheads grow linearly with the ring size n. To circumvent this limitation, we observe linear identities among artifacts produced in the $\mathsf{BRS}_{\mathsf{HE}}$ signing process, which resemble the sum argument relation that underpins the NIZKAoK-based [7] succinct DualRing construction [57, Algorithm 5]. Thereby, our second generic construction $\mathsf{BRS}_{\mathsf{CP}}$ follows a similar approach and is presented in Figure 3. The rationale behind its signing process is detailed as follows.

Upon closer inspection of Equation 3.1, the linearity in $c' = \sum_{j\in[n]} c_j + \sum_{j\in[n]} \beta_j$ enables a more efficient blinding strategy: rather than blinding each c_j with its own β_j, we can pick an arbitrary $l \in [n]$ (potentially $l = i$) and fold all blinding factors into a single term $\beta = \sum_{j\in[n]} \beta_j$ attached only to c_l. That is, we may regard $\beta_l = \beta$, and $\beta_j = 0$ for any $j \in [n] \wedge j \neq l$. Then, for the signer to derive the challenge for index i, the relation can be expressed as:

$$\begin{aligned} c_i &:= c' - \sum_{j\in[n]\wedge j\neq i} c_j = c + \beta - c_l - \sum_{j\in[n]\wedge j\neq i\wedge j\neq l} c_j \\ &= c - (c_l - \beta) - \sum_{j\in[n]\wedge j\neq i\wedge j\neq l} c_j. \end{aligned} \tag{3.2}$$

This derivation suggests having the user blind pk_l with $-\beta$, and hence $R' := R + \mathsf{LF}.\mathrm{F}(\alpha) - \beta \cdot pk_l$. To prevent forgeries, the index l must be chosen *deterministically* as a function of the ring $\mathbf{pk}$, *e.g.*, via a hash-based lexicographical ordering of the public keys.

Let the signer compute its response as before, *i.e.*, $z := r - c_i \cdot sk_i$. By comparing R' with the LF evaluation at the point $z' = z + \alpha$, we obtain the following identity [8]:

$$R' - \mathsf{LF}.\mathrm{F}(z') = (c_l - \beta) \cdot pk_l + \sum_{j\in[n]\wedge j\neq l} c_j \cdot pk_j. \tag{3.3}$$

Note that the LF evaluation at $c_i \cdot sk_i$ incurs an analogous correction term when $\mathsf{LF}.\Psi \not\equiv 0$ (cf. Remark 2); thus, we also restrict it to be identically zero for $\mathsf{BRS}_{\mathsf{CP}}$.

The last piece of our discussion concerns how to compute the pseudo challenge c. A naïve choice of $\mathrm{H}(\mathbf{pk}, R', m)$ fails to bind β due to the linearity in $c' = c + \beta$ (and $c_l + \beta$). Indeed, the user may pick some $m^* \neq m$, compute $c^* = \mathrm{H}(\mathbf{pk}, R', m^*)$, and then set $\beta^* = c' - c^*$. This effectively produces a forgery on message m^* under the same c'. On the other hand, to preserve blindness, the signer must not learn β and c' simultaneously, which rules out hashing β in the clear, *e.g.*, $\mathrm{H}(\mathbf{pk}, R', m, \beta)$ as in $\mathsf{BRS}_{\mathsf{HE}}$. These requirements, together with the involvement of NIZKAoK (*i.e.* NISA), suggests adopting the commit-and-prove (CP) paradigm [12] over the commitment of β and the sum argument relation.

Concretely, let $\mathsf{COM} = (\mathrm{Setup}, \mathrm{Commit}, \mathrm{Verify})$ be a commitment scheme, and denote the pair of commitment and opening for β by $(R_\beta, O_\beta) \leftarrow \mathsf{COM}.\mathrm{Commit}(ck, \beta)$, where $ck \leftarrow \mathsf{COM}.\mathrm{Setup}(1^\lambda)$. The pseudo challenge is set to $c := \mathrm{H}(\mathbf{pk}, R', m, R_\beta)$, and the CP-enhanced sum argument relation, denoted by $\mathcal{R}_{\mathsf{CPSA}}$, is defined as follows. For any instance $x := (\mathbf{pk}, R', R_\beta, z')$ and witness $w := (\{c_j\}_{j\in[n]}, \beta, O_\beta)$, $\mathcal{R}_{\mathsf{CPSA}}(x, w) = 1$ if and only if:

$$\begin{aligned} &\text{Equation 3.3} \wedge \mathrm{H}(\mathbf{pk}, R', m, R_\beta) = (c_l - \beta) + \textstyle\sum_{j\in[n]\wedge j\neq l} c_j \\ &\qquad \wedge \mathsf{COM}.\mathrm{Verify}(ck, R_\beta, \beta, O_\beta) = 1. \end{aligned} \tag{3.4}$$

We further denote the NIZKAoK protocol tailored to $\mathcal{R}_{\mathsf{CPSA}}$ as $\Pi_{\mathsf{CPSA}} = (\mathrm{Setup}, \mathrm{Prove}, \mathrm{Verify})$ (cf. Section 2.4).

Putting these pieces together, our second construction $\mathsf{BRS}_{\mathsf{CP}}$ meets the BRS requirements by combining the LF-based backbone with COM and Π_{CPSA}.

Security analysis. We formally state the security guarantee, and the proof is deferred to Section B.2.

Theorem 2 (Security of $\mathsf{BRS}_{\mathsf{CP}}$). *Let LF be an LF family, H be a hash function modeled as a random oracle, COM be a commitment scheme, and Π_{CPSA} be a NIZKAoK. The construction $\mathsf{BRS}_{\mathsf{CP}}$ in Figure 3 satisfies the*

[7] It is referred to as a non-interactive sum argument (NISA) protocol in *loc. cit.*.

[8] Combining Equation 3.3 with a slightly rephrased Equation 3.2, *i.e.*, $c = (c_l - \beta) + \sum_{j\in[n]\wedge j\neq l} c_j$, yields a form of the sum argument relation of [57] (recalled in Equation 3.5). However, this alone is not sufficient for our purposes; the full relation is deferred to Equation 3.4.

Algorithm $\mathsf{BRS_{CP}}.\mathrm{Setup}(1^\lambda)$:
00 $\mathsf{LF.pp} \leftarrow \mathsf{LF.PGen}(1^\lambda)$
01 Let $\mathrm{H} : \{0,1\}^* \to \mathcal{S}$ be a hash function
02 $ck \leftarrow \mathsf{COM.Setup}(1^\lambda)$
03 $(\mathrm{crs}, \tau) \leftarrow \Pi_{\mathsf{CPSA}}.\mathrm{Setup}(1^\lambda, \mathcal{R}_{\mathsf{CPSA}})$
04 $\mathrm{pp} := (\mathsf{LF.pp}, \mathrm{H}, ck, \mathrm{crs})$
05 **return** (pp, τ)
// Parse $(\lambda, \mathcal{S}, \mathcal{D}, \mathcal{R}, \mathrm{H}, ck, \mathrm{crs}) \in \mathrm{pp}$

$\mathsf{BRS_{CP}}.\mathrm{KeyGen}(\mathrm{pp})$:
06 $sk \leftarrow_\$ \mathcal{D}; pk \leftarrow \mathsf{LF.F}(sk)$
07 **return** (sk, pk)

$\mathsf{BRS_{CP}}.\mathrm{Sign}_1(sk_i, \mathbf{pk})$:
08 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$
09 $r \leftarrow_\$ \mathcal{D}; c_j \leftarrow_\$ \mathcal{S}, \forall j \in [n] \wedge j \neq i$
10 $R := \mathsf{LF.F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j$
11 **return** $(R, \mathsf{state_S})$

$\mathsf{BRS_{CP}}.\mathrm{User}_1(\mathbf{pk}, R, m)$:
12 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$
13 $\alpha \leftarrow_\$ \mathcal{D}; \beta \leftarrow_\$ \mathcal{S}$
14 $R' := R + \mathsf{LF.F}(\alpha) - \beta \cdot pk_l$
15 $(R_\beta, O_\beta) \leftarrow \mathsf{COM.Commit}(ck, \beta)$
16 $c := \mathrm{H}(\mathbf{pk}, R', m, R_\beta); c' := c + \beta$
17 **return** $(c', \mathsf{state_U})$

$\mathsf{BRS_{CP}}.\mathrm{Sign}_2(\mathsf{state_S}, c')$:
18 Parse $(sk_i, r, \{c_j\}_{j\in[n]\wedge j\neq i})$ from $\mathsf{state_S}$
19 $c_i := c' - \sum_{j\in[n]\wedge j\neq i} c_j$
20 $z := r - c_i \cdot sk_i$
21 **return** $(\{c_j\}_{j\in[n]}, z)$

$\mathsf{BRS_{CP}}.\mathrm{User}_2(\mathsf{state_U}, (\{c_j\}_{j\in[n]}, z))$:
22 Parse $(\mathbf{pk}, \alpha, \beta, R', R_\beta, O_\beta, c')$ from $\mathsf{state_U}$
23 **return** $\perp$ if $c' \neq \sum_{j\in[n]} c_j$
24 $z' := z + \alpha$
25 $\pi \leftarrow \Pi_{\mathsf{CPSA}}.\mathrm{Prove}((\mathbf{pk}, R', R_\beta, z'), (\{c_j\}_{j\in[n]}, \beta, O_\beta))$
26 **return** $\sigma := (R', R_\beta, z', \pi)$

$\mathsf{BRS_{CP}}.\mathrm{Verify}(\mathbf{pk}, m, \sigma)$:
27 Parse $\sigma = (R', R_\beta, z', \pi)$
28 **return** 0 if $\Pi_{\mathsf{CPSA}}.\mathrm{Verify}((\mathbf{pk}, R', R_\beta, z'), \pi) = 0$
29 **return** 1

Fig. 3: $\mathsf{BRS_{CP}}$: A Generic BRS from LF and CP-NIZKAoK

following properties in the random oracle model: correctness; $(k, k+1)$*-unforgeability w.r.t. insider corruption if* LF *is collision resistant,* COM *is binding, and* Π_{CPSA} *is knowledge sound; signer-anonymity against full key exposure; and statistical blindness if* COM *is perfect hiding and* Π_{CPSA} *is statistical zero-knowledge.*

3.4 Concrete Instantiations

Now, we provide concrete instantiations of our generic constructions in the discrete logarithm (DL)-based setting (Definition 15). The resulting fully specified BRS schemes serve as the basis for the construction (Section 4) and implementation (Section 5) of iToken.

Let $(\mathbb{G}, \cdot)$ be a cyclic group of prime order p. For convenience, we write the group law multiplicatively throughout this section, with identity denoted by $1_\mathbb{G}$.

The LF-based backbone: Okamoto-Schnorr [40]. An LF family $\mathsf{LF} = (\mathrm{PGen}, \mathrm{F}, \Psi)$ with $\Psi \equiv 0$ underpins both generic construction. Under the DL assumption, the instantiation based on Okamoto-Schnorr [40] satisfies the required properties, as shown in [29]. For completeness, we recall the construction below.

- $\mathsf{LF.PGen}(1^\lambda)$ outputs $\mathrm{pp} := (\mathbb{G}, p, g_1, g_2)$, where $g_1, g_2 \in \mathbb{G}$ are generators. The sets defined by pp are $\mathcal{S} := \mathbb{Z}_p$, $\mathcal{D} := \mathbb{Z}_p^2$, and $\mathcal{R} := \mathbb{G}$. For $s \in \mathbb{Z}_p$, $(x_1, x_2) \in \mathbb{Z}_p^2$, and $z \in \mathbb{G}$, $s \cdot (x_1, x_2) := (sx_1, sx_2) \in \mathbb{Z}_p^2$ and $s \cdot z := z^s \in \mathbb{G}$.
- $\mathsf{LF.F}((x_1, x_2))$ outputs $g_1^{x_1} g_2^{x_2}$ for $(x_1, x_2) \in \mathbb{Z}_p^2$.

Beyond Okamoto–Schnorr, LF can be instantiated from the module short integer solution problem [37], yielding a post-quantum BRS candidate. However, as noted in [29, Section 1.3], the unforgeability reduction relies on LF possessing perfect correctness, which may not be strictly satisfied by lattice-based instantiations. We therefore defer a detailed exploration to future work.

Remark 3 (On the applicability to Schnorr-type signatures). It should be noted that the plain Schnorr [48] does not qualify as an LF family due to the absence of a torsion-free element from the kernel. Nevertheless, our methodologies

introduced for lifting a blind signature to the ring setting, namely the integration of an HE scheme or a CP-NIZKAoK protocol, remain applicable [9] to Schnorr-type blind signatures with additive blinding (*i.e.* $c' = c + \beta$). However, as observed by [23], the security reductions for such concrete constructions are qualitatively distinct from the LF-based modular treatments of [29], a distinction that similarly applies to our BRS case.

HE in $\mathsf{BRS}_{\mathsf{HE}}$: Castagnos-Laguillaumie (CL) encryption [13]. As the algebraic setting of $\mathsf{BRS}_{\mathsf{HE}}$ is fixed by the Okamoto-Schnorr-based LF, it suffices to instantiate an HE scheme HE=(KeyGen, Enc, Dec, Eval) that supports homomorphic addition and scalar multiplication over $\mathcal{M} = \mathbb{Z}_p$. We use CL encryption [13], which leverages a decisional Diffie–Hellman (DDH) group equipped with a subgroup where the DL problem is efficiently solvable.

Definition 14. *A* DDH group with an easy DL subgroup *is defined by a pair of algorithms* $(\mathrm{Gen}, \mathrm{Solve})$*. The group generation algorithm* Gen *takes as input two parameters* λ *and* μ*, and outputs a tuple* $(B, n, p, s, g, f, \mathbb{G}, \mathbb{F})$*. Here,* p *is a* μ*-bit integer,* s *is a* λ*-bit integer,* $\gcd(p, s) = 1$*,* $n = p \cdot s$*, and* B *is an upper bound for* s*. The set* $(G, \cdot)$ *is a cyclic group of order* n *generated by* g*, and* $\mathbb{F} \subset \mathbb{G}$ *is the subgroup of* $\mathbb{G}$ *of order* p *with generator* f*.* B *is chosen s.t. the distribution induced by* $\{g^r, r \leftarrow_{\$} \{0, \ldots, Bp-1\}\}$ *is statistically indistinguishable from the uniform distribution in* $\mathbb{G}$*. We assume that: (1) letting* $\mathbb{H} := \mathbb{G}/\mathbb{F}$*, the canonical surjection* $\pi : \mathbb{G} \to \mathbb{H}$ *is efficiently computable from the description of* $(\mathbb{G}, \mathbb{H}, p)$*, and (2) any* $h \in \mathbb{H}$ *can be efficiently lifted in* $\mathbb{G}$*, i.e., one can compute* $h_\ell \in \pi^{-1}(h)$*. We suppose moreover that:*

- *The DL problem is easy in* $\mathbb{F}$*:* Solve *is a deterministic polynomial time algorithm that solves the DL problem in* $\mathbb{F}$*.*
$$\Pr\left[x' = x \wedge \left| \begin{array}{c} (B, n, p, s, g, f, \mathbb{G}, \mathbb{F}) \leftarrow \mathrm{Gen}(1^\lambda, 1^\mu); \\ x \leftarrow_{\$} \mathbb{Z}_p; X = f^x; \\ x' := \mathrm{Solve}(B, p, g, f, \mathbb{G}, \mathbb{F}, X) \end{array} \right.\right] = 1$$
- *The DDH problem is hard in* $\mathbb{G}$*: for any PPT adversary* $\mathcal{A}$ *with access to* Solve*, the following probability is* $\mathsf{negl}(\lambda)$*.*
$$\left| \Pr\left[b' = b \wedge \left| \begin{array}{c} (B, n, p, s, g, f, \mathbb{G}, \mathbb{F}) \leftarrow \mathrm{Gen}(1^\lambda, 1^\mu); \\ x, y, z \leftarrow_{\$} \mathbb{Z}_n; X = g^x; Y = g^y; \\ b \leftarrow_{\$} \{0,1\}; Z_0 = g^z; Z_1 = g^{xy}; \\ b' \leftarrow \mathcal{A}(B, p, g, f, \mathbb{G}, \mathbb{F}, X, Y, Z_b, \mathrm{Solve}(\cdot)) \end{array} \right.\right] - \frac{1}{2} \right|$$

The construction is recalled below, with minor adjustments to align with the syntax in Section 2.2.

- $\mathsf{HE}.\mathrm{KeyGen}(1^\lambda)$ runs $(B, n, p, s, g, f, \mathbb{G}, \mathbb{F}) \leftarrow \mathrm{Gen}(1^\lambda, 1^\mu)$, samples $x \leftarrow_{\$} \{0, \ldots, Bp-1\}$ (since n is unknown in the sequel), sets $h = g^x$, and outputs $(ek, dk) := ((B, p, g, h, f), x)$.
- $\mathsf{HE}.\mathrm{Enc}(ek, m)$ samples $r \leftarrow_{\$} \{0, \ldots, Bp-1\}$, computes $\mathrm{ct}_1 = g^r, \mathrm{ct}_2 = f^m h^r$, and outputs $\mathrm{ct} := (\mathrm{ct}_1, \mathrm{ct}_2)$.
- $\mathsf{HE}.\mathrm{Eval}(ek, f, \mathrm{ct}_1, \mathrm{ct}_2)$ is only defined for addition $+(\cdot,\cdot)$ and scalar multiplication $\cdot(s, \cdot)$ (*w.r.t.* a scalar s):
  - $\mathsf{HE}.\mathrm{Eval}(ek, +(\cdot,\cdot), \mathrm{ct}_1, \mathrm{ct}_2)$ parses $\mathrm{ct}_1 = (\mathrm{ct}_{11}, \mathrm{ct}_{12})$ and $\mathrm{ct}_2 = (\mathrm{ct}_{21}, \mathrm{ct}_{22})$. It computes $\mathrm{ct}'_1 = \mathrm{ct}_{11}\mathrm{ct}_{21}$ and $\mathrm{ct}'_2 = \mathrm{ct}_{12}\mathrm{ct}_{22}$, samples $r \leftarrow_{\$} \{0, \ldots, Bp-1\}$, and outputs $\mathrm{ct} := (\mathrm{ct}'_1 g^r, \mathrm{ct}'_2 h^r)$.
  - $\mathsf{HE}.\mathrm{Eval}(ek, \cdot(s, \cdot), \mathrm{ct})$ parses $\mathrm{ct} = (\mathrm{ct}_1, \mathrm{ct}_2)$, computes $\mathrm{ct}'_1 = \mathrm{ct}_1^s$ and $\mathrm{ct}'_2 = \mathrm{ct}_2^s$, samples $r \leftarrow_{\$} \{0, \ldots, Bp-1\}$, and outputs $\mathrm{ct} := (\mathrm{ct}'_1 g^r, \mathrm{ct}'_2 h^r)$.
- $\mathrm{Dec}(dk, \mathrm{ct})$ parses $dk = x, \mathrm{ct} = (\mathrm{ct}_1, \mathrm{ct}_2)$, computes $M = \mathrm{ct}_2/\mathrm{ct}_1^x$, and outputs $m := \mathrm{Solve}(B, p, g, f, \mathbb{G}, \mathbb{F}, M)$.

COM and Π_{CPSA} in BRS_1: Pedersen commitment [42] and NISA [57]. By hashing R_β into the pseudo challenge c, the blinding factor β is fixed for the challenge computation while remaining hidden. Since our BRS serves as a building block for anonymous tokens, perfectly hiding is preferable [43]. We therefore instantiate COM with the Pedersen commitment, which satisfies computational binding and perfect hiding under the DL assumption [42].

- $\mathsf{COM}.\mathrm{Setup}(1^\lambda)$ samples $h \leftarrow_{\$} \mathbb{G} \setminus \{1_\mathbb{G}\}$ and outputs $ck := (\mathbb{G}, p, g, h)$, where $g \in \mathbb{G}$ is a generator.
- $\mathsf{COM}.\mathrm{Commit}(ck, m)$ samples $t \leftarrow_{\$} \mathbb{Z}_p$ and outputs a commitment and opening pair $(R, O) := (g^m h^t, t)$.
- $\mathsf{COM}.\mathrm{Verify}(ck, R, m, O)$ outputs 1 if $R = g^m h^O$, or 0 otherwise.

[9] We demonstrate this applicability with a concrete BRS construction derived from an ROS-resilient Schnorr-type blind signature in Figure 4.

To present our (Pedersen-based) Π_{CPSA}, we first recall the sum argument relation of [57], for which an efficient protocol is proposed via an adaptation of the inner product argument of [10]. For an instance $(\mathbf{g} = (g_1, \ldots, g_n) \in \mathbb{G}^n, P \in \mathbb{G}, c \in \mathbb{Z}_p)$ and a witness $\mathbf{a} = (a_1, \ldots, a_n) \in \mathbb{Z}_p^n$, the relation is defined by:

$$P = \prod_{i\in[n]} g_i^{a_i} \wedge c = \sum_{i\in[n]} a_i. \tag{3.5}$$

For comparison, we restate $\mathcal{R}_{\mathsf{CPSA}}$ (Equation 3.4) in multiplicative form and replace COM.Verify with its condition:

$$\begin{aligned} R' \cdot (\mathsf{LF.F}(z'))^{-1} &= \textstyle\prod_{i\in[n]} pk_i^{c_i} \cdot pk_l^{-\beta} \\ \wedge \mathrm{H}(\mathbf{pk}, R', m, R_\beta) &= \textstyle\sum_{i\in[n]} c_i - \beta \wedge R_\beta = g^\beta h^{O_\beta}. \end{aligned} \tag{3.6}$$

Observe that the production $\prod_{i\in[n]} pk_i^{c_i} \cdot pk_l^{-\beta}$ and the commitment $g^\beta h^{O_\beta}$ can be padded with identity elements:

$$R' \cdot (\mathsf{LF.F}(z'))^{-1} = \prod_{i\in[n]} pk_i^{c_i} \cdot pk_l^{-\beta} \cdot 1_{\mathbb{G}}^{O_\beta/2} \cdot 1_{\mathbb{G}}^{-O_\beta/2} \text{ and}$$

$$R_\beta = g^\beta \cdot h^t = \prod_{i\in[n]}^{n} 1_{\mathbb{G}}^{c_i} \cdot (g^{-1})^{-\beta} \cdot h^{O_\beta/2} \cdot (h^{-1})^{-O_\beta/2},$$

where $O_\beta/2$ is shorthand for $O_\beta \cdot 2^{-1} \in \mathbb{Z}_p$ (since p is odd). Accordingly, the relation in Equation 3.6 can be viewed as holding with $\mathrm{H}(\mathbf{pk}, R', m, R_\beta) = \sum_{i\in[n]} c_i + (-\beta) + O_\beta/2 + (-O_\beta/2)$. This allows $\mathcal{R}_{\mathsf{CPSA}}$ to be expressed as a standard sum argument over the product group $\widehat{\mathbb{G}} = \mathbb{G} \times \mathbb{G}$, to which the NISA protocol [57, Algorithm 4] applies directly (recalled in Section A.4). Concretely, the instance is given by $(\widehat{\mathbf{g}}, P, c)$, where:

$$\widehat{\mathbf{g}} = \big((pk_1, 1), \ldots, (pk_n, 1),\ (pk_l, g^{-1}),\ (1, h),\ (1, h^{-1})\big) \in \widehat{\mathbb{G}}^{n+3},$$

$$P = \big(R' \cdot (\mathsf{LF.F}(z'))^{-1}, R_\beta\big) \in \widehat{\mathbb{G}},\ \text{and } c = \mathrm{H}(m, \mathbf{pk}, R', R_\beta) \in \mathbb{Z}_p;$$

and the witness is $\mathbf{a} = (c_1, \ldots, c_n,\ -\beta,\ O_\beta/2,\ -O_\beta/2) \in \mathbb{Z}_p^{n+3}$.

3.5 Discussion: The Reduction Limitation, Practical Implications, and Alternative Constructions

The LF-based canonical three-move construction inherits the same exponential loss in the unforgeability reduction (as a function of the number of *concurrently invoked signing sessions*) as in [29]. As further highlighted by [7,52,32], this loss is not merely a limitation of the proof technique, but reflects a fundamental vulnerability to ROS attacks that exploit the linearity of the signer's response. We refer to these works for a detailed discussion.

In practice, the reduction loss implies that the security guarantee of our LF-based BRS constructions is meaningful only when each ring member participates in at most polylogarithmically many concurrently opened signing sessions. While a larger ring can increase aggregate throughput by spreading sessions across n members, it does not remove the underlying per-signer concurrency bound. System-level mitigations include: (1) stateful admission control, where each signer tracks and restricts the number of concurrently opened sessions, and (2) key evolution with forward security, where each signer periodically rotates its key pairs and the verification is performed *w.r.t.* the ring for the corresponding epoch.

At the same time, the blind signature literature provides generic approaches for improving concurrency tolerance, most notably the cut-and-choose paradigm [33] and its parallel-instance refinements [15]. These techniques amplify security by embedding the base scheme into parallel sessions and randomly opening (most of) them for consistency checks, at the cost of additional communication and computation overhead.

A different direction is to build BRS from a three-move blind signature scheme that is designed to resist ROS-based forgeries, *e.g.*, the ones proposed in [52]. These constructions admit a security reduction to a ROS variant called weighted fractional ROS, which is proved to have an exponential, unconditional lower bound. In comparison, the plain ROS problem can be solved in polynomial time for certain parameter regimes [7].

In particular, as noted in Remark 3, we apply our CP-NIZKAoK-based approach to the ROS-resilient Schnorr-type blind signature construction of [52, Figure 4], the BS_1 scheme (recalled in Section A.2). Accordingly, the resulting concrete BRS construction, denoted by BRS_1, is given in Figure 4. While we fully specify BRS_1, we defer an end-to-end security analysis to future work to maintain focus on the LF-based framework [10].

In BRS_1, COM is instantiated by the Pedersen commitment and the CP-NIZKAoK protocol Π_{CPSA_1} is tailored to the following CP-enhanced relation $\mathcal{R}_{\mathsf{CPSA}_1}$. For any instance $x := (\mathbf{pk}, R_1', R_\beta, y', z')$ and witness $w :=$

[10] As mentioned in Remark 3, a security proof for BRS_1 may require the generic group model inherited from BS_1, and is therefore be qualitatively different from our LF-based analysis.

$(\{c_j\}_{j\in[n]}, \beta, \gamma, O_\beta, y)$, $\mathcal{R}_{\mathsf{CPSA}_1}(x, w) = 1$ if and only if:

$$R'_1 \cdot g^{-z'} = \left(\prod_{j\in[n]\wedge j\neq i} pk_j^{c_j} \cdot pk_i^{c_i\cdot y}\right)^{\gamma} \cdot \left(pk_l^{y'}\right)^{-\beta}$$
$$\wedge \mathrm{H}(\mathbf{pk}, R'_1, pk_l^{y'}, m, R_\beta) = {y'}^{-1} \cdot \gamma \cdot \left(\textstyle\sum_{j\in[n]\wedge j\neq i} c_j + c_i \cdot y\right) - \beta$$
$$\wedge R_\beta = g^\beta h^{O_\beta}.$$

The sum argument (and hence correctness) is preserved by Line 20:

$$\gamma \cdot \left(\sum_{j\in[n]\wedge j\neq i} c_j + c_i \cdot y\right) = \gamma \cdot y \cdot c' = y' \cdot c'$$
$$= y' \cdot \left(\mathrm{H}(\mathbf{pk}, R'_1, pk_l^{y'}, m, R_\beta) + \beta\right).$$

$\mathsf{BRS}_1.\mathrm{Setup}(1^\lambda)$:
00 Let $(\mathbb{G}, p, g)$ be a group
01 Let $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_p$ be a hash function
02 $h \leftarrow \mathsf{COM}.\mathrm{Setup}(1^\lambda; (\mathbb{G}, p, g))$
// COM.Setup is overload to reuse $\mathbb{G}$
03 $(\mathrm{crs}, \tau) \leftarrow \Pi_{\mathsf{CPSA}_1}.\mathrm{Setup}(1^\lambda, \mathcal{R}_{\mathsf{CPSA}})$
04 $\mathrm{pp} := (\mathbb{G}, p, g, \mathbb{Z}_p, \mathrm{H}, h, \mathrm{crs})$
05 **return** (pp, τ)

$\mathsf{BRS}_1.\mathrm{KeyGen}(\mathrm{pp})$:
06 $sk \leftarrow_\$ \mathbb{Z}_p^*; pk \leftarrow g^s k$
07 **return** (sk, pk)

$\mathsf{BRS}_1.\mathrm{Sign}_1(sk_i, \mathbf{pk})$:
08 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$
09 $r \leftarrow_\$ \mathbb{Z}_p; y \leftarrow_\$ \mathbb{Z}_p^*$
10 $c_j \leftarrow_\$ \mathbb{Z}_p, \forall j \in [n] \wedge j \neq i$
11 $R_1 := g^r \cdot \prod_{j\in[n]\wedge j\neq i} pk_j^{c_j}; R_2 := pk_l^y$
// l is chosen deterministically
12 **return** $((R_1, R_2)), \mathsf{state}_\mathrm{S})$

$\mathsf{BRS}_1.\mathrm{User}_1(\mathbf{pk}, R, m)$:
13 Parse $\mathbf{pk} = \{pk_j\}_{j\in[n]}$ and $R = (R_1, R_2)$
14 $\alpha, \beta \leftarrow_\$ \mathbb{Z}_p; \gamma \leftarrow_\$ \mathbb{Z}_p^*; R'_2 := R_2^\gamma$
15 $R'_1 := g^\alpha \cdot R_1^\gamma \cdot {R'_2}^{-\beta}$
16 $t \leftarrow_\$ \mathbb{Z}_p; (R_\beta, O_\beta) := (g^\beta h^t, t)$
17 $c := \mathrm{H}(\mathbf{pk}, R'_1, R'_2, m, R_\beta); c' := c + \beta$
18 **return** $(c', \mathsf{state}_\mathrm{U})$

$\mathsf{BRS}_1.\mathrm{Sign}_2(\mathsf{state}_\mathrm{S}, c')$:
19 Parse $(sk_i, y, \{c_j\}_{j\in[n]\wedge j\neq i})$ from $\mathsf{state}_\mathrm{S}$
20 $c_i := c' - y^{-1} \cdot \sum_{j\in[n]\wedge j\neq i} c_j$
21 $z := r - c_i \cdot y \cdot sk_i$
22 **return** $(\{c_j\}_{j\in[n]}, y, z)$

$\mathsf{BRS}_1.\mathrm{User}_2(\mathsf{state}_\mathrm{U}, (\{c_j\}_{j\in[n]}, y, z))$:
23 Parse $(\mathbf{pk}, R_2, \alpha, \beta, \gamma, R'_1, R_\beta, O_\beta, c')$ from $\mathsf{state}_\mathrm{U}$
24 **return** $\perp$ if $y = 0 \vee R_2 \neq pk_l^y \vee c' \neq \sum_{j\in[n]} c_j$
25 $y' := \gamma \cdot y; z' := \gamma \cdot z + \alpha$
26 $\pi \leftarrow \Pi_{\mathsf{CPSA}_1}.\mathrm{Prove}((\mathbf{pk}, R'_1, pk_l^{y'}, R_\beta, y', z'), (\{c_j\}_{j\in[n]}, \beta, \gamma, O_\beta, y))$
// Note that $pk_l^{y'} = pk_l^{y\gamma} = R'_2$
27 **return** $\sigma := (R'_1, R_\beta, y', z', \pi)$

$\mathsf{BRS}_1.\mathrm{Verify}(\mathbf{pk}, m, \sigma)$:
28 Parse $\sigma = (R'_1, R_\beta, y', z', \pi)$
29 **return** 0 if $y' = 0 \vee$
$\Pi_{\mathsf{CPSA}_1}.\mathrm{Verify}((\mathbf{pk}, R'_1, pk_l^{y'}, R_\beta, y', z'), \pi) = 0$
30 **return** 1

Fig. 4: BRS_1: A BRS Construction with (potentially) Exponential Security based on [52, Figure 4] and CP-NIZKAoK Π_{CPSA_1}

4 One-Time-Use Anonymous Token with Issuance Hiding

A one-time-use anonymous token (OTAT) scheme enables a *user* to obtain a *token* from an *issuer*; the user can, in turn, redeem this token with a *verifier* as a trust signal for one-time authentication. The fundamental security notions are *(one-more) unforgeability* and *unlinkability*. Unforgeability guarantees that no adversary can redeem more tokens than were validly issued; whereas, unlinkability protects the user's privacy by ensuring that no adversary (including malicious issuers and verifiers, even if they collude) can link a token redemption to a particular issuance session. While issuer-hiding appears in the broader anonymous credentials literature, it remains understudied in the context of OTAT. Existing definitions typically prevent malicious verifiers from identifying the issuer only in the redemption phase, leaving the issuer unprotected during issuance. To bridge this gap, we propose the stronger notion of *issuance hiding*, which guarantees issuer anonymity against any adversary (including malicious users) throughout the entire token lifecycle, *i.e.*, from the initial issuance session to final redemption.

4.1 Syntax and Security Notions

To simultaneously support issuance hiding and unlinkability, we adapt the three-move structure from our revisited BRS formalization to establish a compatible OTAT syntax, departing from the traditional two-move, user-initiated model [36,17].

Formally, an OTAT scheme consists of a tuple of algorithms $\mathsf{AT} = (\text{Setup}, \text{KeyGen}, \langle \text{Issue}, \text{Obtain} \rangle, \text{Redeem})$. While the syntax mostly mirrors that of our BRS scheme, we extend the issuance protocol to explicitly capture public metadata and adopt the standard terminology found in the OTAT literature.

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: On input the security parameter λ, the setup algorithm outputs the public parameter pp.
- $(sk_i, pk_i) \leftarrow \text{KeyGen}(i)$: On input a issuer index $i \in \mathbb{N}$, the key generation algorithm outputs a secret-public key pair (sk_i, pk_i).
- $(b, \sigma) \leftarrow \langle \text{Issue}(sk_i, \mathbf{pk}, M), \text{Obtain}(\mathbf{pk}, M, t) \rangle$ is an interactive protocol between an issuer holding a secret key sk_i (*s.t.* the corresponding $pk_i \in \mathbf{pk}$) and a user possessing an attribute string $t \in \{0,1\}^\lambda$. Both parties mutually agree on a pair $(\mathbf{pk}, M)$, where $\mathbf{pk} = \{pk_j\}_{j \in [n]}$ denotes the "ring" of eligible issuers and M denotes the public metadata. The three-move, issuer-initiated protocol is specified as follows.
  - $(R, \mathsf{state}_\text{I}) \leftarrow \text{Issue}_1(sk_i, \mathbf{pk}, M)$: Run by the issuer, the algorithm outputs a commitment R and the issuer's state state_I.
  - $(c, \mathsf{state}_\text{O}) \leftarrow \text{Obtain}_1(\mathbf{pk}, M, R, t)$: Run by the user, the algorithm outputs a challenge c and the user's state state_O.
  - $z \leftarrow \text{Issue}_2(\mathsf{state}_\text{I}, c)$: On input the issuer's state_I and a challenge c, the algorithm outputs a response z deterministically.
  - $\sigma \leftarrow \text{Obtain}_2(\mathsf{state}_\text{O}, z)$: On input the user's state_O and the response z, this algorithm outputs a token σ deterministically.

  The protocol additionally outputs a bit b: $b = 0$ if any algorithm aborts or $\sigma = \perp$; or $b = 1$ otherwise. Moreover,
- $b \leftarrow \text{Redeem}(\mathbf{pk}, M, t, \sigma)$: The deterministic redemption algorithm outputs a bit $b \in \{0,1\}$: $b = 1$ if σ is a valid token on the attribute pair (M, t) *w.r.t.* the ring $\mathbf{pk}$, or $b = 0$ otherwise.

An OTAT scheme is required to satisfy correctness, one-more unforgeability, issuance hiding, and unlinkability. By interpreting $\mathbf{pk}, M$ as a single concatenated input $\mathbf{pk}||M$ and renaming Sign, User, Verify to Issue, Obtain, Redeem, respectively (with the same renaming applied to oracles, *e.g.*, SignLoR to IssueLoR), the formal definitions of OTAT properties follow directly from the BRS notions of correctness (Definition 10), unforgeability (Definition 11), signer-anonymity (Definition 12), and blindness (Definition 13). We provide the rationale for this adaptation by comparing the resulting security notions with the established OTAT formalizations [36,17], highlighting both commonalities and differences.

- *Unforgeability.* The three-move issuance protocol (as opposed to the two-move one in *loc. cit.*) requires unforgeability to account for concurrency. The notion inherited from BRS preserves the one-more structure and provides concurrency guarantees. Additionally, it tolerates insider corruption, which is inherent to the issuer-side ring structure introduced for issuance hiding.
- *Issuance hiding.* The notion inherits the signer-anonymity of BRS, which allows explicit modeling of malicious user participation during issuance, and is formulated under a full key-exposure model. Together, these features provide strictly stronger guarantees than the traditional issuer-hiding property.
- *Unlinkability.* The notion inherits the blindness of BRS. As is standard, our "1-out-of-2" formulation is equivalent, via a hybrid argument, to the "1-out-of-many" definition considered in *loc. cit.*.

4.2 Our iToken from BRS

Our iToken is constructed by implementing its algorithms using the corresponding components of a BRS scheme, instantiated via BRS_HE (Figure 2), BRS_CP (Figure 3), or BRS_1 (Figure 4). For completeness, we provide the construction as follows. Parse $\mathsf{BRS} \in \{\mathsf{BRS}_\mathsf{HE}, \mathsf{BRS}_\mathsf{CP}, \mathsf{BRS}_1\}$:

- $\text{iToken.Setup}(1^\lambda)$: It invokes $\text{pp} \leftarrow \mathsf{BRS}.\text{Setup}(1^\lambda)$ and returns the public parameter pp.
- $\text{iToken.KeyGen}(\text{pp})$: It invokes $(sk, pk) \leftarrow \mathsf{BRS}.\text{KeyGen}(\text{pp})$ and returns the key pair (sk, pk).
- $\langle \text{iToken.Issue}(sk_i, \mathbf{pk}, M), \text{iToken.Obtain}(\mathbf{pk}, M, t) \rangle$: The interactive issuance protocol is depicted in Figure 5.
- $\text{iToken.Redeem}(\mathbf{pk}, M, t, \sigma)$: It invokes $b \leftarrow \mathsf{BRS}.\text{Verify}(\mathbf{pk}||M, t, \sigma)$ and returns b.

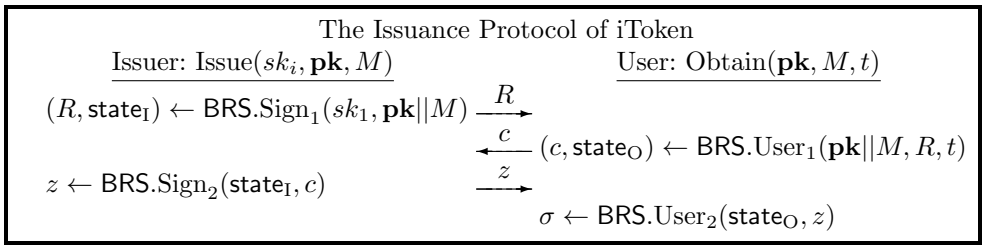


Fig. 5: The Issuance Protocol of iToken.

Its security guarantee is stated in Theorem 3 and follows directly from the security of the underlying BRS scheme, proven in Theorem 1 and 2. We therefore omit the proof of Theorem 3.

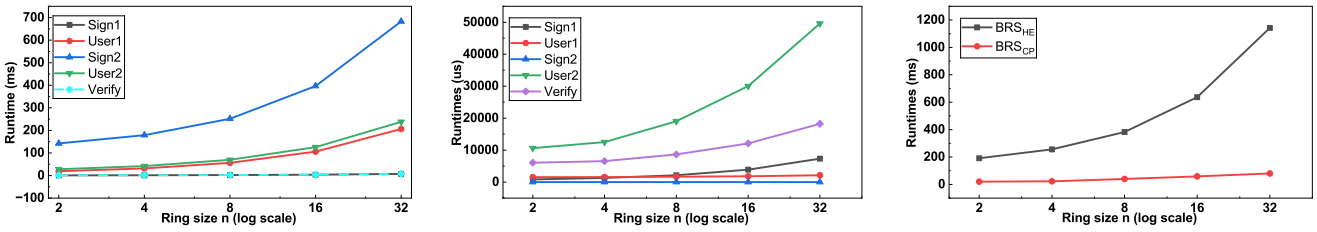
Theorem 3 (Security of iToken). *Let* BRS *be a BRS scheme. The construction of iToken satisfies the following properties:* correctness *if* BRS *is correct;* one-more unforgeability *if* BRS *satisfies* $(k, k+1)$*-unforgeability w.r.t. insider corruption;* issuance hiding *if* BRS *satisfies signer-anonymity against full key exposure; and unlinkability if* BRS *satisfies signer blindness.*

5 Implementation and Evaluation

Table 2: Communication Complexity Performance Comparison: PMBT [36] vs. ATHM [17] vs. HinToken-I [53] vs. Our iToken.

Scheme	Sign_0	User_0	Sign_1	User_1	Verify	Token Size (Byte)
PMBT [36]	–	38.9 µs	235.7 µs	292.5 µs	77.1	448 B
ATHM [17]	–	37.0 µs	311.4 µs	346.8 µs	56.1	384 B
HinToken-I [53]	–	158.1 µs	1683.1 µs	3318.2 µs	4440.6 µs	1008 B
iToken via $\mathsf{BRS}_{\mathsf{HE}}$ $(n=4)$	1290 µs	31606 µs	178837 µs	42046 µs	1559 µs	192 B
iToken via $\mathsf{BRS}_{\mathsf{HE}}$ $(n=8)$	2170 µs	56567 µs	251625 µs	69962 µs	2466 µs	320 B
iToken via $\mathsf{BRS}_{\mathsf{CP}}$ $(n=4)$	1292 µs	1594 µs	3.7 µs	12515 µs	6572 µs	359 B
iToken via $\mathsf{BRS}_{\mathsf{CP}}$ $(n=8)$	2160 µs	1680 µs	4.8 µs	19509 µs	8683 µs	425 B

- Benchmarks for HinToken (on BLS12-381), and related schemes (on curve25519-dalek).
- Benchmarks for iToken (on secp256r1, ring size equals to 4 and 8).



(a) $\mathsf{BRS}_{\mathsf{HE}}$ **(b)** $\mathsf{BRS}_{\mathsf{HE}}$ **(c)** Total runtime

Fig. 6: Runtime comparison as a function of the ring size.

We implemented our $\mathsf{BRS}_{\mathsf{HE}}$ and $\mathsf{BRS}_{\mathsf{CP}}$ constructions as described in Section 3.4 in C++ using the Botan cryptographic library. Our implementation is instantiated over the secp256r1 elliptic curve with SHA-256 as the underlying hash function. All group elements are represented in compressed form, and public keys follow an Okamoto–Schnorr style with two secret scalars per signer. All benchmarks were conducted on a machine equipped with an Intel® Core™i7-13700K CPU (up to 5.4 GHz). We exclude key generation from the reported results, as it is performed only once and does not affect the amortized performance of token issuance and verification.

As shown in Table 2, we compare our iToken instantiated with different ring sizes ($n \in \{4, 8\}$) against the PMBT scheme [36] and the ATHM scheme [17], as well as the issuer-hiding OTAT scheme HinToken [53]. The

results confirm that issuer-hiding OTAT schemes generally incur higher computational costs than non-issuer-hiding constructions. In particular, both PMBT and ATHM outperform issuer-hiding OTAT schemes across all phases. Among issuer-hiding constructions, HinToken-I exhibits a faster issuance phase than our schemes. We emphasize that this performance gap arises primarily from differences in the underlying building blocks. In particular, HinToken relies on SPS-EQ, which are computationally lighter than BRS-based constructions. Specifically, $\mathsf{BRS}_{\mathsf{HE}}$ relies on CL encryption, which incurs substantial computational overhead. In contrast, $\mathsf{BRS}_{\mathsf{CP}}$ requires the generation of a ring signature together with a CPSA proof, resulting in an inherent $\mathcal{O}(n)$ computational complexity with respect to the ring size. The results further show that the computation cost of both $\mathsf{BRS}_{\mathsf{HE}}$ and $\mathsf{BRS}_{\mathsf{CP}}$ increases as the ring size grows from $n = 4$ to $n = 8$, which is consistent with their expected complexity trend. Regarding token size, $\mathsf{BRS}_{\mathsf{HE}}$ produces the shortest tokens, $\mathsf{BRS}_{\mathsf{CP}}$ achieves a moderate token size, and HinToken yields the largest tokens among the evaluated schemes. For our constructions, the token size grows with the ring size, reflecting the inclusion of additional group elements. This behavior is consistent with the token structure: when the ring size is small, $\mathsf{BRS}_{\mathsf{HE}}$ minimizes auxiliary proof material, whereas $\mathsf{BRS}_{\mathsf{CP}}$ incurs additional group elements to support its proof components. Consequently, our schemes offer a favorable trade-off between computation cost and token size, particularly for small ring sizes.

Figure 6a and Figure 6b illustrate the runtime of $\mathsf{BRS}_{\mathsf{HE}}$ and $\mathsf{BRS}_{\mathsf{CP}}$, respectively, as the ring size increases. We intentionally present the two constructions in separate plots due to the substantially different computational costs of their underlying cryptographic primitives. While both schemes scale approximately linearly with ring size, $\mathsf{BRS}_{\mathsf{HE}}$ incurs significantly larger constant factors due to encryption-based components, whereas $\mathsf{BRS}_{\mathsf{CP}}$ achieves lower latency by avoiding CL encryption and relying on CPSA proofs. Figure 6c further summarizes the total runtime comparison between $\mathsf{BRS}_{\mathsf{HE}}$ and $\mathsf{BRS}_{\mathsf{CP}}$.

6 Conclusion

This paper explores the possibility of designing an issuance hiding OTAT to mitigate token reuse and targeted DoS attacks, *e.g.*, the Kuaishou incident. Therefore, the crux of achieving this goal lies in embedding the issuer into a resilient and collective anonymity group (*i.e.*, multiple issuers) at *issuance time*. This ensures that no attackers, including malicious users and verifiers, can identify which specific authority issued a given token, even if the attacker participates in the issuance session. However, achieving issuance hiding while simultaneously offering strong user privacy renders the existing *blind-then-ring* pattern insufficient. We therefore turn to BRS, which inherently follows a *blind-and-ring* pattern: the ring structure already present in the blind signing interaction. While BRS has appeared in prior work, its formalization is upgraded here to meet modern requirements. We then provide two generic BRS constructions, each accompanied by a rigorous security analysis establishing its security. Furthermore, we instantiate these constructions and present our iToken from BRS. Finally, we successfully executed a proof-of-concept demonstration of the BRS and iToken, and our experiments indicate that BRS and iToken are deployable and are currently being used effectively.

References

1. Abe, M., Okamoto, T.: Provably secure partially blind signatures. In: Bellare, M. (ed.) CRYPTO 2000. LNCS, vol. 1880, pp. 271–286. Springer, Berlin, Heidelberg (Aug 2000). https://doi.org/10.1007/3-540-44598-6_17
2. Amjad, G., Yeo, K., Yung, M.: RSA blind signatures with public metadata. Proc. Priv. Enhancing Technol. **2025**(1), 37–57 (2025)
3. Backendal, M., Bellare, M., Sorrell, J., Sun, J.: The fiat-shamir zoo: Relating the security of different signature variants. In: Gruschka, N. (ed.) Secure IT Systems - 23rd Nordic Conference, NordSec 2018, Oslo, Norway, November 28-30, 2018, Proceedings. Lecture Notes in Computer Science, vol. 11252, pp. 154–170. Springer (2018). https://doi.org/10.1007/978-3-030-03638-6_10, https://doi.org/10.1007/978-3-030-03638-6_10
4. Baldimtsi, F., Hanzlik, L., Nguyen, Q., Yadav, A.: Non-interactive anonymous tokens with private metadata bit. Cryptology ePrint Archive, Report 2025/430 (2025), https://eprint.iacr.org/2025/430
5. Benarroch, D., Campanelli, M., Fiore, D., Kim, J., Lee, J., Oh, H., Querol, A.: Proposal: Commit-and-prove zero-knowledge proof systems and extensions (2024), https://docs.zkproof.org/pages/standards/accepted-workshop4/proposal-commit.pdf
6. Bender, A., Katz, J., Morselli, R.: Ring signatures: Stronger definitions, and constructions without random oracles. In: Halevi, S., Rabin, T. (eds.) TCC 2006. LNCS, vol. 3876, pp. 60–79. Springer, Berlin, Heidelberg (Mar 2006). https://doi.org/10.1007/11681878_4

7. Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. Journal of Cryptology **35**(4), 25 (Oct 2022). https://doi.org/10.1007/s00145-022-09436-0
8. Benhamouda, F., Lepoint, T., Orrù, M., Raykova, M.: Publicly verifiable anonymous tokens with private metadata bit. Cryptology ePrint Archive, Report 2022/004 (2022), https://eprint.iacr.org/2022/004
9. Bosk, D., Frey, D., Gestin, M., Piolle, G.: Hidden issuer anonymous credential. PoPETs **2022**(4), 571–607 (Oct 2022). https://doi.org/10.56553/popets-2022-0123
10. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE Symposium on Security and Privacy. pp. 315–334. IEEE Computer Society Press (May 2018). https://doi.org/10.1109/SP.2018.00020
11. Camenisch, J., Piveteau, J.M., Stadler, M.: Blind signatures based on the discrete logarithm problem (rump session). In: De Santis, A. (ed.) EUROCRYPT'94. LNCS, vol. 950, pp. 428–432. Springer, Berlin, Heidelberg (May 1995). https://doi.org/10.1007/BFb0053458
12. Canetti, R., Lindell, Y., Ostrovsky, R., Sahai, A.: Universally composable two-party and multi-party secure computation. In: 34th ACM STOC. pp. 494–503. ACM Press (May 2002). https://doi.org/10.1145/509907.509980
13. Castagnos, G., Laguillaumie, F.: Linearly homomorphic encryption from DDH. In: Nyberg, K. (ed.) CT-RSA 2015. LNCS, vol. 9048, pp. 487–505. Springer, Cham (Apr 2015). https://doi.org/10.1007/978-3-319-16715-2_26
14. Catalano, D., Fiore, D.: Using linearly-homomorphic encryption to evaluate degree-2 functions on encrypted data. In: Ray, I., Li, N., Kruegel, C. (eds.) ACM CCS 2015. pp. 1518–1529. ACM Press (Oct 2015). https://doi.org/10.1145/2810103.2813624
15. Chairattana-Apirom, R., Hanzlik, L., Loss, J., Lysyanskaya, A., Wagner, B.: PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part III. LNCS, vol. 13509, pp. 3–31. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_1
16. Chan, T.K., Fung, K., Liu, J.K., Wei, V.K.: Blind spontaneous anonymous group signatures for ad hoc groups. In: Castelluccia, C., Hartenstein, H., Paar, C., Westhoff, D. (eds.) Security in Ad-hoc and Sensor Networks, First European Workshop, ESAS 2004, Heidelberg, Germany, August 6, 2004, Revised Selected Papers. Lecture Notes in Computer Science, vol. 3313, pp. 82–94. Springer (2004). https://doi.org/10.1007/978-3-540-30496-8_8, https://doi.org/10.1007/978-3-540-30496-8_8
17. Chase, M., Durak, F.B., Vaudenay, S.: Anonymous tokens with stronger metadata bit hiding from algebraic MACs. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part II. LNCS, vol. 14082, pp. 418–449. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38545-2_14
18. Chaum, D.: Blind signatures for untraceable payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) CRYPTO'82. pp. 199–203. Plenum Press, New York, USA (1982). https://doi.org/10.1007/978-1-4757-0602-4_18
19. Crites, E.C., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Snowblind: A threshold blind signature in pairing-free groups. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part I. LNCS, vol. 14081, pp. 710–742. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38557-5_23
20. Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy pass: Bypassing internet challenges anonymously. PoPETs **2018**(3), 164–180 (Jul 2018). https://doi.org/10.1515/popets-2018-0026
21. Duong, D.H., Susilo, W., Tran, H.T.N.: A multivariate blind ring signature scheme. Comput. J. **63**(8), 1194–1202 (2020). https://doi.org/10.1093/COMJNL/BXZ128, https://doi.org/10.1093/comjnl/bxz128
22. Feige, U., Shamir, A.: Zero knowledge proofs of knowledge in two rounds. In: Brassard, G. (ed.) CRYPTO'89. LNCS, vol. 435, pp. 526–544. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_46
23. Fuchsbauer, G., Plouviez, A., Seurin, Y.: Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 63–95. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_3
24. Gajland, P., Janneck, J., Kiltz, E.: Ring signatures for deniable AKEM: Gandalf's fellowship. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part I. LNCS, vol. 14920, pp. 305–338. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68376-3_10
25. Garg, S., Gupta, D.: Efficient round optimal blind signatures. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 477–495. Springer, Berlin, Heidelberg (May 2014). https://doi.org/10.1007/978-3-642-55220-5_27
26. Ghadafi, E.: Sub-linear blind ring signatures without random oracles. In: Stam, M. (ed.) 14th IMA International Conference on Cryptography and Coding. LNCS, vol. 8308, pp. 304–323. Springer, Berlin, Heidelberg (Dec 2013). https://doi.org/10.1007/978-3-642-45239-0_18
27. Ghadafi, E.: Efficient round-optimal blind signatures in the standard model. In: Kiayias, A. (ed.) FC 2017. LNCS, vol. 10322, pp. 455–473. Springer, Cham (Apr 2017). https://doi.org/10.1007/978-3-319-70972-7_26
28. Hanff, K., Lehmann, A., Özbay, C.: Security analysis of privately verifiable privacy pass. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025. pp. 2922–2936. ACM (2025)
29. Hauck, E., Kiltz, E., Loss, J.: A modular treatment of blind signatures from identification schemes. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 345–375. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_12

30. Juels, A., Luby, M., Ostrovsky, R.: Security of blind digital signatures (extended abstract). In: Kaliski, Jr., B.S. (ed.) CRYPTO'97. LNCS, vol. 1294, pp. 150–164. Springer, Berlin, Heidelberg (Aug 1997). https://doi.org/10.1007/BFb0052233
31. Karantaidou, I., Renawi, O., Baldimtsi, F., Kamarinakis, N., Katz, J., Loss, J.: Blind multisignatures for anonymous tokens with decentralized issuance. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 1508–1522. ACM Press (Oct 2024). https://doi.org/10.1145/3658644.3690364
32. Katsumata, S., Lai, Y.F., Reichle, M.: Breaking parallel ROS: Implication for isogeny and lattice-based blind signatures. In: Tang, Q., Teague, V. (eds.) PKC 2024, Part I. LNCS, vol. 14601, pp. 319–351. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57718-5_11
33. Katz, J., Loss, J., Rosenberg, M.: Boosting the security of blind signature schemes. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 468–492. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_16
34. Katz, J., Sefranek, M.: Issuer hiding for bbs-based anonymous credentials. IACR Cryptol. ePrint Arch. p. 1080 (2025), https://eprint.iacr.org/2025/2080
35. Kiltz, E., Masny, D., Pan, J.: Optimal security proofs for signatures from identification schemes. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 33–61. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_2
36. Kreuter, B., Lepoint, T., Orrù, M., Raykova, M.: Anonymous tokens with private metadata bit. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part I. LNCS, vol. 12170, pp. 308–336. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56784-2_11
37. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. DCC **75**(3), 565–599 (2015). https://doi.org/10.1007/s10623-014-9938-4
38. Lu, X., Au, M.H., Zhang, Z.: Raptor: A practical lattice-based (linkable) ring signature. In: Deng, R.H., Gauthier-Umaña, V., Ochoa, M., Yung, M. (eds.) ACNS 2019. LNCS, vol. 11464, pp. 110–130. Springer, Cham (Jun 2019). https://doi.org/10.1007/978-3-030-21568-2_6
39. Mohammed, E., Emarah, A.E., El-Shennaway, K.: A blind signature scheme based on elgamal signature. In: Proceedings of the Seventeenth National Radio Science Conference. 17th NRSC'2000 (IEEE Cat. No.00EX396). pp. C25/1–C25/6 (2000). https://doi.org/10.1109/NRSC.2000.838954
40. Okamoto, T.: Provably secure and practical identification schemes and corresponding signature schemes. In: Brickell, E.F. (ed.) CRYPTO'92. LNCS, vol. 740, pp. 31–53. Springer, Berlin, Heidelberg (Aug 1993). https://doi.org/10.1007/3-540-48071-4_3
41. Okamoto, T.: Efficient blind and partially blind signatures without random oracles. In: Halevi, S., Rabin, T. (eds.) TCC 2006. LNCS, vol. 3876, pp. 80–99. Springer, Berlin, Heidelberg (Mar 2006). https://doi.org/10.1007/11681878_5
42. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) CRYPTO'91. LNCS, vol. 576, pp. 129–140. Springer, Berlin, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_9
43. Pointcheval, D., Sanders, O.: Short randomizable signatures. In: Sako, K. (ed.) CT-RSA 2016. LNCS, vol. 9610, pp. 111–126. Springer, Cham (Feb / Mar 2016). https://doi.org/10.1007/978-3-319-29485-8_7
44. Pointcheval, D., Stern, J.: Provably secure blind signature schemes. In: Kim, K., Matsumoto, T. (eds.) ASIACRYPT'96. LNCS, vol. 1163, pp. 252–265. Springer, Berlin, Heidelberg (Nov 1996). https://doi.org/10.1007/BFb0034852
45. Rivest, R.L., Shamir, A., Tauman, Y.: How to leak a secret. In: Boyd, C. (ed.) ASIACRYPT 2001. LNCS, vol. 2248, pp. 552–565. Springer, Berlin, Heidelberg (Dec 2001). https://doi.org/10.1007/3-540-45682-1_32
46. Rosenberg, M., White, J.D., Garman, C., Miers, I.: zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure. In: 2023 IEEE Symposium on Security and Privacy. pp. 790–808. IEEE Computer Society Press (May 2023). https://doi.org/10.1109/SP46215.2023.10179430
47. Sanders, O., Traoré, J.: Compact issuer-hiding authentication, application to anonymous credential. PoPETs **2024**(3), 645–658 (Jul 2024). https://doi.org/10.56553/popets-2024-0097
48. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) CRYPTO'89. LNCS, vol. 435, pp. 239–252. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_22
49. Schnorr, C.P.: Security of blind discrete log signatures against interactive attacks. In: Qing, S., Okamoto, T., Zhou, J. (eds.) ICICS 01. LNCS, vol. 2229, pp. 1–12. Springer, Berlin, Heidelberg (Nov 2001). https://doi.org/10.1007/3-540-45600-7_1
50. Silde, T., Strand, M.: Anonymous tokens with public metadata and applications to private contact tracing. In: Proc. FC 2022. vol. 13411, pp. 179–199
51. Silde, T., Strand, M.: Anonymous tokens with public metadata and applications to private contact tracing. In: Eyal, I., Garay, J.A. (eds.) FC 2022. LNCS, vol. 13411, pp. 179–199. Springer, Cham (May 2022). https://doi.org/10.1007/978-3-031-18283-9_9
52. Tessaro, S., Zhu, C.: Short pairing-free blind signatures with exponential security. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 782–811. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_27

53. Tu, A., Jia, M., He, K., Chen, J., Du, R.: Anonymous tokens with designated-reader metadata bit. In: 35th USENIX Security Symposium, USENIX Security 2026. USENIX Association (2026)
54. Wagner, B.L.U.: Techniques for highly efficient and secure interactive signatures (2024). https://doi.org/http://dx.doi.org/10.22028/D291-42072
55. Wu, Q., Zhang, F., Susilo, W., Mu, Y.: An efficient static blind ring signature scheme. In: Won, D., Kim, S. (eds.) Information Security and Cryptology - ICISC 2005, 8th International Conference, Seoul, Korea, December 1-2, 2005, Revised Selected Papers. Lecture Notes in Computer Science, vol. 3935, pp. 410–423. Springer (2005). https://doi.org/10.1007/11734727_32, https://doi.org/10.1007/11734727_32
56. Xie, Y.Y., Chen, X.B., Yang, Y.X.: A new lattice-based blind ring signature for completely anonymous blockchain transaction systems. Security and Communication Networks **2022**(1), 4052029 (2022)
57. Yuen, T.H., Esgin, M.F., Liu, J.K., Au, M.H., Ding, Z.: DualRing: Generic construction of ring signatures with efficient instantiations. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part I. LNCS, vol. 12825, pp. 251–281. Springer, Cham, Virtual Event (Aug 2021). https://doi.org/10.1007/978-3-030-84242-0_10

A Supplementary Materials

This section supplements Section 2 and Section 3 with additional details. We also present a linear function (LF) family-based representation for the DualRing construction [57].

Additional notations. For vectors (of size n): $\mathbf{a}_{[:l]}$ and $\mathbf{a}_{[l:]}$ denote $(a_1, \ldots, a_l)$ and $(a_{l+1}, \ldots, a_n)$, respectively; $\mathbf{a} \circ \mathbf{b}$ is the Hadamard product $(a_1b_1, a_2b_2, \ldots a_nb_n)$, and $\langle \mathbf{a}, \mathbf{b} \rangle$ is the inner product $\sum_{i\in[n]} a_ib_i$; and $\mathbf{a}^b$, $\mathbf{a} + b$, and $\mathbf{a}^{\mathbf{b}}$ represent $(a_1^b, a_2^b, \ldots, a_n^b)$, $(a_1 + b, a_2 + b, \ldots, a_n + b)$, and $\prod_{i\in[n]} a_i^{b_i}$, respectively.

A.1 Auxiliary Definitions

We recall the discrete logarithm (DL) assumption, which underpins the Okamoto–Schnorr-based LF family instantiation, the Pedersen commitment, and our CP-enhanced NIZKAoK construction.

Definition 15 (DL Assumption). *Let $\mathbb{G}$ be a cyclic group of prime order p with bit length λ, and let $g \in \mathbb{G}$ be a generator. The DL assumption holds if, for any PPT adversary $\mathcal{A}$, the following probability is $\mathsf{negl}(\lambda)$ for any λ.*

$$\Pr\left[h = g^x | h \leftarrow_{\$} \mathbb{G}; x \leftarrow \mathcal{A}(\mathbb{G}, p, g, h)\right]$$

Moreover, we formalize the semi-honest signer and malicious signer blindness of BRS.

Definition 16 (Semi-Honest Signer Blindness). *A BRS scheme satisfies semi-honest signer blindness if, for any PPT adversary $\mathcal{A}(\mathcal{A}_1, \mathcal{A}_2)$ can access $\mathcal{O} := (\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_1}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_2}^{sess_0,sess_1,b})$, the following probability is $\mathsf{negl}(\lambda)$ for any λ and $\text{pp} \leftarrow \text{Setup}(1^\lambda)$.*

$$\left| \Pr\left[b' = b \;\middle|\; \begin{array}{c} \{r_i\}_{i\in[n]} \leftarrow \mathcal{A}_1(\text{pp}) \\ \{(sk_i, pk_i) := \text{KeyGen}(\text{pp}; r_i)\}_{i\in[n]} \\ (m_0, m_1, \mathsf{state}) \leftarrow \mathcal{A}_1(\{(sk_i, pk_i)\}_{i\in[n]}) \\ b \leftarrow_{\$} \{0,1\}; b' \leftarrow \mathcal{A}_2^{\mathcal{O}}(\mathsf{state}) \end{array} \right] - \frac{1}{2} \right|,$$

where the input to $\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}$ is $(\mathbf{pk} := \{pk_i\}_{i\in[n]}, m_0, m_1)$.

Definition 17 (Malicious Signer Blindness). *A BRS scheme satisfies malicious signer blindness if, for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ can access $\mathcal{O} := (\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_1}^{sess_0,sess_1,b}, \mathcal{O}_{\text{User}_2}^{sess_0,sess_1,b})$, the following probability is $\mathsf{negl}(\lambda)$ for any λ and $\text{pp} \leftarrow \text{Setup}(1^\lambda)$.*

$$\left| \Pr\left[b' = b \;\middle|\; \begin{array}{c} (\{(sk_i, pk_i)\}_{i\in[n]}\, m_0, m_1, \mathsf{state}) \leftarrow \mathcal{A}_1(\text{pp}) \\ b \leftarrow_{\$} \{0,1\}; b' \leftarrow \mathcal{A}_2^{\mathcal{O}}(\mathsf{state}) \end{array} \right] - \frac{1}{2} \right|,$$

where the input to $\mathcal{O}_{\text{Init}}^{sess_0,sess_1,b}$ is $(\mathbf{pk} := \{pk_i\}_{i\in[n]}, m_0, m_1)$.

A.2 Canonical Three-Move Blind Signatures

A canonical three-move blind signature consists of a tuple of algorithms $(\text{Setup}, \text{KeyGen}, \langle \text{Sign}, \text{User} \rangle, \text{Verify})$, where the interactive signing protocol is further specified by $(\text{Sign}_1, \text{User}_1, \text{Sign}_2, \text{User}_2)$. We refer to [29] for detailed syntax and security definitions. This section recalls the constructions that inspire our BRS design.

Algorithm $\mathsf{BS}.\mathrm{Setup}(1^\lambda)$:
00 $\mathsf{LF}.\mathrm{pp} \leftarrow \mathsf{LF}.\mathrm{PGen}(1^\lambda)$
01 Let $\mathrm{H} : \{0,1\}^* \to \mathcal{S}$ be a hash function
02 **return** $\mathrm{pp} := (\mathsf{LF}.\mathrm{pp}, \mathrm{H})$
// Parse $(\lambda, \mathcal{S}, \mathcal{D}, \mathcal{R}, \mathrm{H}) \in \mathrm{pp}$

Algorithm $\mathsf{BS}.\mathrm{KeyGen}(\mathrm{pp})$:
03 $sk \leftarrow_\$ \mathcal{D}; pk \leftarrow \mathsf{LF}.\mathrm{F}(sk)$
04 **return** (sk, pk)

Algorithm $\mathsf{BS}.\mathrm{Sign}_1(sk)$:
05 $r \leftarrow_\$ \mathcal{D}; R := \mathsf{LF}.\mathrm{F}(r)$
06 **return** $(R, \mathsf{state}_\mathrm{S})$

Algorithm $\mathsf{BS}.\mathrm{User}_1(pk, R, m)$:
07 $\alpha \leftarrow_\$ \mathcal{D}; \beta \leftarrow_\$ \mathcal{S}$
08 $R' := R + \mathsf{LF}.\mathrm{F}(\alpha) + \beta \cdot pk$
09 $c := \mathrm{H}(R', m); c' := c + \beta$
10 **return** $(c', \mathsf{state}_\mathrm{U})$

Algorithm $\mathsf{BS}.\mathrm{Sign}_2(\mathsf{state}_\mathrm{S}, c')$:
11 Parse (sk, r) from $\mathsf{state}_\mathrm{S}$
12 $z := r + c' \cdot sk$
13 **return** z

Algorithm $\mathsf{BS}.\mathrm{User}_2(\mathsf{state}_\mathrm{U}, z)$:
14 Parse $(pk, R, m, \alpha, \beta, R', c')$ from $\mathsf{state}_\mathrm{U}$
15 **return** $\perp$ if $\mathsf{LF}.\mathrm{F}(z) \neq R + c' \cdot pk$
16 $c := \mathrm{H}(R', m); z' := z + \alpha + \mathsf{LF}.\Psi(pk, -c, c')$
17 **return** $\sigma := (c, z')$

Algorithm $\mathsf{BS}.\mathrm{Verify}(pk, m, \sigma)$:
18 Parse $\sigma = (c, z')$
19 $R' := \mathsf{LF}.\mathrm{F}(z') - c \cdot pk$
20 **return** 0 if $c \neq \mathrm{H}(R', m)$
21 **return** 1

Fig. 7: BS: The LF-Based Construction of [29]

The LF-based canonical three-move construction. Let $\mathsf{LF} = (\mathrm{PGen}, \mathrm{F}, \Psi)$ be an LF family as given in Section 2.1. The LF-based canonical three-move construction of [29], denoted by BS, is provided in Figure 7.

The Schnorr-style construction with exponential security. Let $(\mathbb{G}, p, g)$ represent a cyclic group of prime order p with generator g. The Schnorr-style blind signature construction of [52], denoted by BS_1, that reduces to the weighted fractional ROS problem and thus achieves exponential security, is given in Figure 8.

Algorithm $\mathsf{BS}_1.\mathrm{Setup}(1^\lambda)$:
00 Given $(\mathbb{G}, p, g)$:
01 Let $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_p$ be a hash function
02 **return** $\mathrm{pp} := (\mathbb{G}, p, g, \mathrm{H})$

Algorithm $\mathsf{BS}_1.\mathrm{KeyGen}(\mathrm{pp})$:
03 $sk \leftarrow_\$ \mathbb{Z}_p^*; pk \leftarrow g^s k$
04 **return** (sk, pk)

Algorithm $\mathsf{BS}_1.\mathrm{Sign}_1(sk)$:
05 $r \leftarrow_\$ \mathbb{Z}_p; y \leftarrow_\$ \mathbb{Z}_p^*$
06 $R_1 := g^r; R_2 := pk^y$
07 **return** $((R_1, R_2)), \mathsf{state}_\mathrm{S})$

Algorithm $\mathsf{BS}_1.\mathrm{User}_1(pk, R, m)$:
08 Parse $R = (R_1, R_2)$
09 $\alpha, \beta \leftarrow_\$ \mathbb{Z}_p; \gamma \leftarrow_\$ \mathbb{Z}_p^*; R_2' := R_2^\gamma$
10 $R_1' := g^\alpha \cdot R_1^\gamma \cdot {R_2'}^\beta$
11 $c := \mathrm{H}(R_1', R_2', m); c' := c + \beta$
12 **return** $(c', \mathsf{state}_\mathrm{U})$

Algorithm $\mathsf{BS}_1.\mathrm{Sign}_2(\mathsf{state}_\mathrm{S}, c')$:
13 Parse (sk, r, y) from $\mathsf{state}_\mathrm{S}$
14 $z := r + c' \cdot y \cdot sk$
15 **return** (y, z)

Algorithm $\mathsf{BS}_1.\mathrm{User}_2(\mathsf{state}_\mathrm{U}, (y, z))$:
16 Parse $(pk, R_1, R_2, \alpha, \beta, \gamma, c, c')$ from $\mathsf{state}_\mathrm{U}$
17 **return** $\perp$ if $y = 0 \vee R_2 \neq pk^y \vee g^z \neq R_1 \cdot R_2^{c'}$
18 $y' := \gamma \cdot y; z' := \gamma \cdot z + \alpha$
19 **return** $\sigma := (c, y', z')$

Algorithm $\mathsf{BS}_1.\mathrm{Verify}(\mathbf{pk}, m, \sigma)$:
20 Parse $\sigma = (c, y', z')$
21 **return** 0 if $y' = 0 \vee c \neq \mathrm{H}(g^{z'} \cdot pk^{-c \cdot y'}, pk^{y'}, m)$
22 **return** 1

Fig. 8: BS_1: The BS Construction with Exponential Security of [52, Figure 4]

A.3 An LF-Based Representation for DualRing

A ring signature scheme consists of a tuple of algorithms $\mathsf{RS} = (\mathrm{Setup}, \mathrm{KeyGen}, \mathrm{Sign}, \mathrm{Verify})$. We refer to [57] for detailed syntax and security definitions. The DualRing framework [57] gives a generic transform from a Type-T^* canonical identification to an efficient ring signature. We observe that this transform admits an equivalent representation in terms of LF families, which is provided in Figure 9.

Algorithm $\mathsf{RS.Setup}(1^\lambda)$:
00 $\mathsf{LF.pp} \leftarrow \mathsf{LF.PGen}(1^\lambda)$
01 Let $\mathrm{H} : \{0,1\}^* \to \mathcal{S}$ be a hash function
02 **return** $\mathrm{pp} := (\mathsf{LF.pp}, \mathrm{H})$
// Parse $(\lambda, \mathcal{S}, \mathcal{D}, \mathcal{R}, \mathrm{H}) \in \mathrm{pp}$

Algorithm $\mathsf{RS.KeyGen}(\mathrm{pp})$:
03 $sk \leftarrow_\$ \mathcal{D}; pk \leftarrow \mathsf{LF.F}(sk)$
04 **return** (sk, pk)

Algorithm $\mathsf{RS.Sign}(sk_i, \mathbf{pk}, m)$:
05 $r \leftarrow_\$ \mathcal{D}; c_j \leftarrow_\$ \mathcal{S}, \forall j \in [n] \wedge j \neq i$
06 $R := \mathsf{LF.F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j$
07 $c := \mathrm{H}(\mathbf{pk}, R, m); c_i := c - \sum_{j\in[n]\wedge j\neq i} c_j$
08 $z := r - c_i \cdot sk_i$
09 **return** $\sigma := (\{c_j\}_{j\in[n]}, z)$

Algorithm $\mathsf{RS.Verify}(\mathbf{pk}, m, \sigma)$:
10 Parse $\sigma = (\{c_j\}_{j\in[n]}, z)$
11 $R := \mathsf{LF.F}(z) + \sum_{j\in[n]} c_j \cdot pk_j; c = \sum_{j\in[n]} c_j$
12 **return** 0 if $c \neq \mathrm{H}(\mathbf{pk}, R', m)$
13 **return** 1

Fig. 9: RS: An LF-Based Representation for DualRing [57]

A.4 Non-Interactive Sum Argument Protocol

As explained in Section 3.4, our relation $\mathcal{R}_{\mathsf{CPSA}}$ in construction $\mathsf{BRS}_{\mathsf{CP}}$ can be expressed as a standard sum argument over the product group $\widehat{\mathbb{G}} = \mathbb{G} \times \mathbb{G}$. This formulation allows the non-interactive sum argument (NISA) protocol [57, Algorithm 4] to be directly applied to our Π_{CPSA}.

Let $\mathrm{H}_Z, \mathrm{H}'_Z : \{0,1\}^* \to \mathbb{Z}_p$ denote two hash functions; we describe the NISA protocol in Algorithm 1.

Algorithm 1: Π_{CPSA} from NISA [57, Algorithm 4]

Procedure $\Pi_{\mathsf{CPSA}}.\mathrm{Prove}(\mathrm{crs}, x = (\mathbf{g}, P, c), w = \mathbf{a})$
  Run protocol PF on input $(\mathbf{g}, u^{\mathrm{H}'_Z(P,u,c)}, \mathbf{a}, \mathbf{1}^n)$

Procedure $\mathsf{PF}.(\mathbf{g}, \hat{u}, \mathbf{a}, \mathbf{b})$
  // $\mathbf{L}, \mathbf{R}$ are initially empty, but maintained throughout recursion.
  // n is the length of vectors $\mathbf{a}$ and $\mathbf{b}$.
  if $n = 1$ **then**
    return $\pi = (\mathbf{L}, \mathbf{R}, a, b)$
  else
    Compute $n' = n/2$, $c_L = \langle \mathbf{a}_{[:n']}, \mathbf{b}_{[n':]} \rangle \in \mathbb{Z}_p$, and $c_R = \langle \mathbf{a}_{[n':]}, \mathbf{b}_{[:n']} \rangle \in \mathbb{Z}_p$
    $L = \mathbf{g}_{[n':]}^{\mathbf{a}_{[:n']}} \cdot \hat{u}^{c_L} \in \mathbb{G}$ and $R = \mathbf{g}_{[:n']}^{\mathbf{a}_{[n':]}} \cdot \hat{u}^{c_R} \in \mathbb{G}$
    $\mathbf{L} = \mathbf{L} \cup \{L\}$ and $\mathbf{R} = \mathbf{R} \cup \{R\}$
    Compute $x = \mathrm{H}_Z(L, R)$, $\mathbf{g}' = \mathbf{g}_{[:n']}^{x^{-1}} \circ \mathbf{g}_{[n':]}^{x} \in \mathbb{G}^{n'}$, $\mathbf{a}' = x \cdot \mathbf{a}_{[:n']} + x^{-1} \cdot \mathbf{a}_{[n':]} \in \mathbb{Z}_q^{n'}$, and $\mathbf{b}' = x^{-1} \cdot \mathbf{b}_{[:n']} + x \cdot \mathbf{b}_{[n':]} \in \mathbb{Z}_q^{n'}$
    Run protocol PF on input $(\mathbf{g}', \hat{u}, \mathbf{a}', \mathbf{b}')$

Procedure $\Pi_{\mathsf{CPSA}}.\mathrm{Verify}(\mathrm{crs}, x = (\mathbf{g}, P, c), \pi = (\mathbf{L}, \mathbf{R}, a, b))$
  $P' = P \cdot u^{c \cdot \mathrm{H}'_Z(P,u,c)}$
  Compute $x_j = \mathrm{H}_Z(L_j, R_j), \forall j \in [\log_2 n]$
  Compute $y_i = \prod_{j\in[\log_2 n]} x_j^{f(i,j)}, \forall i \in [n]$, where $f(i,j) = \begin{cases} 1 & \text{if the } (i-1)\text{'s } j\text{-th bit is 1} \\ -1 & \text{otherwise} \end{cases}$
  Set $\mathbf{y} = (y_1, \ldots, y_n)$ and $\mathbf{x} = (x_1, \ldots, x_{\log_2 n})$
  if $\mathbf{L}^{\mathbf{x}^2} P' \mathbf{R}^{\mathbf{x}^{-2}} = \mathbf{g}^{a \cdot \mathbf{y}} u^{ab \cdot \mathrm{H}'_Z(P,u,c)}$ **then**
    return 1
  return 0

B Formal Proofs

The proofs omitted from the main body are provided below.

At a high level, our unforgeability proof extends that of [29] to the ring setting. The reduction in *loc. cit.* proceeds via an intermediate identification scheme, first reducing to one-more man-in-the-middle (OMMIM) security and then to the collision resistance of the underlying LF family. Similarly, the DualRing reduction of [57] (adapted to our LF-based representation in Section A.3) relies on an identification-based intermediate satisfying security against special impersonation under key-only attack (SIMP-KOA). Observe that SIMP-KOA is implied by OMMIM [11]; rather than introducing these intermediate identification notions explicitly, we fold the arguments together and provide a direct reduction for both generic constructions to LF collision resistance.

B.1 Proof of Theorem 1

Proof. We recall the statement as follows. Let LF be an LF family, H be a hash function modeled as a random oracle, and HE be an HE scheme. The construction $\mathsf{BRS}_{\mathsf{HE}}$ in Figure 2 satisfies the following properties in the random oracle model: *correctness* (Definition 10); $(k, k+1)$-*unforgeability w.r.t.* insider corruption (Definition 11) if LF is collision resistant; *signer-anonymity* against full key exposure (Definition 12); and *blindness* (Definition 13) if HE satisfies semantic security.

Correctness. Consider a signature $\sigma = (\{c'_j\}_{j\in[n]}, z', \beta)$ output from the an honestly executed signing session with honestly generated $\mathrm{pp} \leftarrow \mathrm{Setup}(1^\lambda)$ and key pairs $(sk_j, pk_j) \leftarrow \mathrm{KeyGen}(j), \forall j \in [n]$, where the signer holds (sk_i, pk_i) and the ring is denoted by $\mathbf{pk} = \{pk_j\}_{j\in[n]}$. We must show that the reconstructed blinded commitment in Verify, $R'^* := \mathsf{LF}.\mathrm{F}(z') + \sum_{j\in[n]} c'_j \cdot pk_j$, matches the value $R' = R + \mathsf{LF}.\mathrm{F}(\alpha) + \sum_{j\in[n]} \beta_j \cdot pk_j$ computed in User_1. This equality further ensures a properly derived pseudo challenge $c^* = \mathrm{H}(\mathbf{pk}, R'', m, \beta)$, which leads to $c'^* = c^* + \beta = \sum_{j\in[n]} c'_j$ and hence, $\mathrm{Verify}(\mathbf{pk}, m, \sigma) = 1$.

Concretely, leveraging $R = \mathsf{LF}.\mathrm{F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j$ in Sign_1 and the expansions in Sign_2: $c'_j = c_j + \beta_j, \forall j \in [n] \wedge j \neq i$; $c'_i = c' - \sum_{j\in[n]\wedge j\neq i} c'_j$; and $z' = z + \alpha = r - (c'_i - \beta_i) \cdot sk_i$, we have:

$$
\begin{aligned}
R'^* &= \mathsf{LF}.\mathrm{F}(z') + \sum_{j\in[n]} c'_j \cdot pk_j \\
&= \mathsf{LF}.\mathrm{F}(r - (c'_i - \beta_i) \cdot sk_i + \alpha) + \sum_{j\in[n]} c'_j \cdot pk_j \\
&= \mathsf{LF}.\mathrm{F}(r) + \mathsf{LF}.\mathrm{F}(\alpha) - (c'_i - \beta_i) \cdot \mathsf{LF}.\mathrm{F}(sk_i) + \sum_{j\in[n]} c'_j \cdot pk_j \\
&= \mathsf{LF}.\mathrm{F}(r) + \mathsf{LF}.\mathrm{F}(\alpha) + \sum_{j\in[n]\wedge j\neq i} (c_j + \beta_j) \cdot pk_j + \beta_i \cdot pk_i \\
&= \mathsf{LF}.\mathrm{F}(r) + \sum_{j\in[n]\wedge j\neq i} c_j \cdot pk_j + \mathsf{LF}.\mathrm{F}(\alpha) + \sum_{j\in[n]} \beta_j \cdot pk_j \\
&= R + \mathsf{LF}.\mathrm{F}(\alpha) + \sum_{j\in[n]} \beta_j \cdot pk_j = R'
\end{aligned}
$$

Unforgeability. Since the unforgeability adversary impersonates a user in the experiment, it naturally generates and holds the key pair (ek, dk) of HE. Therefore, we simplify the reduction by implicitly providing (ek, dk) to all adversaries and working directly with the underlying plaintexts, thereby abstracting away the encryption-decryption operations without loss of generality.

Let $\mathcal{A}$ be an adversary that breaks the unforgeability of $\mathsf{BRS}_{\mathsf{HE}}$. We construct an adversary $\mathcal{B}$ that breaks the collision resistance of LF. Recall that q denotes the number of completed signing sessions; we also denote the number of random oracle queries by q_{H}.

Following the reduction technique of [29], we first define a sequence of auxiliary algorithms $\mathcal{A}'_\ell$ for $\ell \in [q+1]$, where each $\mathcal{A}'_\ell$ wraps the unforgeability adversary $\mathcal{A}$ and is subsequently utilized by the collision resistance adversary

[11] More precisely, as established in [29], OMMIM security [29, Definition 4.2] implies security against impersonation under passive/active attack (IMP-PA/AA) [35, Definition 2.2], which in turn implies security against impersonation under key only attack (IMP-KOA) [35, Definition 2.2]. The SIMP-KOA notion [57, Definition 5] is a stronger variant of IMP-KOA: whereas IMP-KOA requires a single valid transcript, SIMP-KOA mandates the production of two valid transcripts with distinct challenges (yet remains weaker than special soundness [35, Definition 2.4] as it does not require full key extraction). Thereby, SIMP-KOA imposes a "one-more" constraint akin to finding a collision, which is subsumed by the strictly stronger OMMIM security.

$\mathcal{B}$. More concretely, each $\mathcal{A}'_\ell$ receives as input $I := (\mathrm{pp}, \{sk_j\}_{j\in[N]})$, where $\mathrm{pp} \leftarrow \mathrm{Setup}(1^\lambda)$ and $sk_j \in \mathcal{D}, \forall j \in [N]$. It then invokes $\mathcal{A}$ with a random tape $\omega \in \Omega$ and uses a vector $\mathbf{h} \in \mathcal{S}^{q_\mathrm{H}}$ to program the random oracle. Here, Ω denotes the space of random tapes and hence, rendering $\mathcal{A}'_\ell$ deterministic for fixed ω.

The description of $\mathcal{A}'_\ell$ is provided in Figure 10. Under this definition, each wrapper $\mathcal{A}'_\ell, \forall \ell \in [q+1]$ perfectly simulates the oracle environment for $\mathcal{A}$: (1) the inputs $\{sk_j\}_{j\in[N]}$ and $\mathbf{h}$ are uniformly distributed; and (2) the wrapper executes the signing algorithms exactly as protocol specification.

The analysis of $\mathcal{A}'_\ell$ is mostly analogous to that of [29]. We introduce variables to reference the values associated with the valid forgeries produced by $\mathcal{A}$ (from Line 12 to 16). Specifically, consider the output tuple $(\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell)$. The variable $\hat{\boldsymbol{\chi}}_\ell$ represents the extracted pre-image of R'_ℓ corresponding to the ℓ-th valid forgery (tracked by the counter ctr) and is set to:

$$\hat{\boldsymbol{\chi}}_\ell = (\hat{\boldsymbol{\chi}}_\ell[0], \hat{\boldsymbol{\chi}}_\ell[1]) := \left(\{c'_j\}_{j\in[n]}, \hat{\mathbf{z}}'_{ctr} + \sum_{j\in[n]} c'_j \cdot sk_j \right).$$

Note that we additionally outputs $\{c'_j\}_{j\in[n]}$ as $\hat{\boldsymbol{\chi}}_\ell[0]$. This is used to handle a special case tailored to the ring setting (detailed in the analysis of $\mathcal{B}$), which does not exist in the standard treatment of [29]. Moreover, the index $\hat{\mathbf{J}}_\ell$ denotes the random oracle query index vid that corresponds to the ℓ-th forgery.

We fix an execution of $\mathcal{A}'_\ell$ via the input tuple, denoted by $I = (\mathrm{pp}, \{sk_j\}_{j\in[N]})$, the vector $\mathbf{h}$, and the randomness ω. We define the set $\mathcal{W}$ of *successful inputs* for $\mathcal{A}'_\ell$ as the set of all tuples $(I, \omega, \mathbf{h})$ that lead to a successful run of $\mathcal{A}'_\ell$, *i.e.*,

$$\mathcal{W} := \left\{ (I, \omega, \mathbf{h}) \mid \hat{\mathbf{J}}_\ell \neq 0; (\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell) := \mathcal{A}'_\ell(I, \mathbf{h}; \omega) \right\}.$$

Note that $\mathcal{W}$ is independent of the index ℓ, as $\mathcal{A}'_\ell$ wraps the same underlying adversary $\mathcal{A}$: if $\mathcal{A}$ succeeds in the unforgeability game (producing $k+1$ valid forgeries), then $\mathcal{A}'_\ell$ will successfully extract values for all $\ell \in [k+1]$. Consequently, the probability of a tuple belonging to $\mathcal{W}$, *i.e.* $(I, \omega, \mathbf{h}) \in \mathcal{W}$ for $pp \leftarrow \mathrm{Setup}(1^\lambda)$ and $(\{sk_j\}_{j\in[N]}, \omega, \mathbf{h}) \leftarrow\!\!\$\ \mathcal{D}^N \times \Omega \times \mathcal{S}^{q_\mathrm{H}}$:

$$\Pr_{\substack{pp\leftarrow\mathrm{Setup}(1^\lambda),\\ (\{sk_j\}_{j\in[N]},\omega,\mathbf{h})\ \leftarrow\!\!\$\ \mathcal{D}^N\times\Omega\times\mathcal{S}^{q_\mathrm{H}}}} [(I, \omega, \mathbf{h}) \in \mathcal{W}]$$

is exactly the advantage of $\mathcal{A}$ against the unforgeability of $\mathsf{BRS}_{\mathsf{HE}}$ (*i.e.*, the probability formula in Definition 11). The remaining probability analysis follows the identical steps established in [29].

Now, we describe the adversary $\mathcal{B}$; its specification is provided in Figure 11. $\mathcal{B}$ begins by sampling a random index $i^* \leftarrow\!\!\$\ [q+1]$ and executing the wrapper $\mathcal{A}'_{i^*}$ with uniformly sampled secret keys $\{sk_j\}_{j\in[N]}$, a fix random tap ω, and a random vector $\mathbf{h}$. The core of the reduction relies on the forking step (Line 6): $\mathcal{B}$ resample the elements in $\mathbf{h}$ starting from the index $\hat{\mathbf{J}}_{i^*}$ (*i.e.*, retaining the prefix $\mathbf{h}_{[:\hat{\mathbf{J}}_{i^*}-1]}$ while replacing the remainder with fresh values). It then executes $\mathcal{A}'_{i^*}$ again, preserving the original random tape ω but using the modified vector $\mathbf{h}'$.

If $\mathcal{B}$ does not abort in the first run, by the definition of $\mathcal{A}'_{i^*}$, we have $\mathsf{LF.F}(\hat{\boldsymbol{\chi}}_{i^*}[1]) = \mathbf{R}'_{\hat{\mathbf{J}}_{i^*}}$. Since $\mathcal{A}'_{i^*}$ takes as input the identical elements (from $\mathbf{h}$ and $\mathbf{h}'$) for answering the first $\hat{\mathbf{J}}_{i^*}-1$ random oracle queries in both executions, its internal state remains identical up to receiving the $\hat{\mathbf{J}}_{i^*}$-th random oracle query (with $\mathbf{R}'_{\hat{\mathbf{J}}_{i^*}}$ and β). That is, the fork changes the programmed oracle answer $c' = \mathrm{H}(\mathbf{pk}^*, R', m, \beta) + \beta$ except with negligible probability. If the second run also yields a valid extraction at index $\hat{\mathbf{J}}_{i^*}$, we obtain $\mathsf{LF.F}(\hat{\boldsymbol{\chi}}'_{i^*}[1]) = \mathbf{R}'_{\hat{\mathbf{J}}_{i^*}} = \mathsf{LF.F}(\hat{\boldsymbol{\chi}}_{i^*}[1])$. That is:

$$\Pr_{\substack{pp\leftarrow\mathrm{Setup}(1^\lambda),\\ (\hat{\boldsymbol{\chi}}_{i^*}[1],\hat{\boldsymbol{\chi}}'_{i^*}[1])\leftarrow\mathcal{B}(\mathrm{pp})}} \begin{bmatrix} \hat{\boldsymbol{\chi}}_{i^*}[1] \neq \hat{\boldsymbol{\chi}}'_{i^*}[1] \wedge \\ \mathsf{LF.F}(\hat{\boldsymbol{\chi}}_{i^*}[1]) = \mathsf{LF.F}(\hat{\boldsymbol{\chi}}'_{i^*}[1]) \end{bmatrix}$$

is the advantage of $\mathcal{B}$ against the collision resistance of LF. We refer all the concrete possibility analysis, including the analysis for output indices to [29, Section 7.1, Lemma 7.1, and Lemma 7.2].

A distinction from [29] arises here: since β is not required to equal $\sum_{j\in[n]} \beta_j$ in Sign_2, the output of $\mathcal{A}$ (hence $\mathcal{A}'_{i^*}$) may redistribute the coefficients, from $\{c'_j\}$ to $\{c''_j\}$, while preserving the sum constraint $c' = \sum_{j\in[n]} c'_j = \sum_{j\in[n]} c''_j$. We note that any such redistribution either changes the corresponding R' due to the change of $\langle \{c'_j\}_{j\in[n]}, \mathbf{pk} \rangle$. This corresponds to a different random oracle query input, which is covered by the analysis of [29]. Whereas, the inverse event leads to an instant collision for LF:

$$\mathsf{LF.F}(\sum_{j\in[n]} c'_j \cdot sk_j) = \mathsf{LF.F}(\sum_{j\in[n]} c''_j \cdot sk_j).$$

Adversary $\mathcal{A}'_\ell(I = (\text{pp}, \{sk_j\}_{j\in[N]}), \mathbf{h}; \omega)$:
00 Parse $(\mathbf{r}, \mathbf{c})$ from ω
01 $\mathbf{pk} := \{\mathsf{LF.F}(sk_j)\}_{j\in[N]}$
02 $sid, vid, q, ctr := 0; S, V := \emptyset$
03 $(\mathbf{pk}^*, \mathbf{m}^*, \boldsymbol{\sigma}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Corrupt}}, \mathcal{O}_{\text{Sign}_1}, \mathcal{O}_{\text{Sign}_2}, \text{H}}(\mathbf{pk})$
04 **return** $\perp$ if $\mathbf{pk}^* \not\subseteq \mathbf{pk} \setminus C$
05 $\mathbf{R}, \mathbf{R}', \mathbf{c}', \mathbf{z} := \emptyset$
06 For $l \in [k+1]$ do:
07 Parse $\boldsymbol{\sigma}^*_l = (\{c'_j\}_{j\in[n]}, z', \beta)$
08 $\mathbf{R}'_l := \mathsf{LF.F}(z') + \langle \{c'_j\}_{j\in[n]}, \mathbf{pk}^* \rangle$
09 $(vid, c') := \text{H}(\mathbf{pk}^*, \mathbf{R}'_l, \mathbf{m}^*_l, \beta); c := \mathbf{h}_{vid}$
10 **return** $\perp$ if $vid \notin V$
11 If $\sum_{j\in[n]} c'_j = c' = c + \beta$:
12 $ctr := ctr + 1$
13 $\hat{\mathbf{z}}'_{ctr} := z'$
14 $\hat{\mathbf{h}}_{ctr} := \mathbf{h}_{vid}$
15 $\hat{\boldsymbol{\chi}}_{ctr} := \left(\{c'_j\}_{j\in[n]}, \hat{\mathbf{z}}'_{ctr} + \sum_{j\in[n]} c'_j \cdot sk_j\right)$
16 $\hat{\mathbf{J}}_{ctr} := vid$
17 $V := V \setminus \{vid\}$
18 $q_{\text{H}} := vid$
19 **return** $(\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell)$ if $ctr \geq q + 1$
20 **return** $(\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell) := (0, 0)$

Procedure $\mathcal{O}^C_{\text{Corrupt}}(i)$:
21 **return** sk_i

Procedure $\mathcal{O}^{sid,S}_{\text{Sign}_1}(i, \mathbf{pk})$:
22 **return** $\perp$ if $pk_i \neq \mathbf{pk}$
23 $sid := sid + 1; S := S \cup \{sid\}$
24 $\mathbf{c}'_{sid}, \mathbf{z}_{sid} := \perp$
25 [a] $\mathbf{R}_{sid} := \mathsf{LF.F}(\mathbf{r}_{sid}) + \langle \mathbf{c}_{sid|\mathbf{pk}}, \mathbf{pk} \rangle$
26 **return** $(sid, \mathbf{R}_{sid})$

Procedure $\mathcal{O}^{S,q}_{\text{Sign}_2}(sid, ch)$:
27 **return** $\perp$ if $sid \notin S$
28 Parse $(i, \mathbf{pk})$ from state_{sid} inherited from $\mathcal{O}_{\text{Sign}_1}$
29 Parse $\mathbf{c}_{sid|\mathbf{pk}} = \{c_j\}_{j\in[n]}$ and $ch = (\{\beta_j\}_{j\in[n]}, c')$
// All elements in ch are under encryption
30 $\mathbf{c}'_{sid} := (\{\beta_j\}_{j\in[n]}, c')$
31 $c'_j := c_j + \beta_j, \forall j \in [n] \wedge j \neq i$
32 $c'_i := c' - \sum_{j\in[n]\wedge j\neq i} c'_j$
33 $z := \mathbf{r}_{sid} - (c'_i - \beta_i) \cdot sk_i$
34 $S := S \setminus \{sid\}; q := q + 1$
35 **return** $\mathbf{z}_{sid} := (\{c'_j\}_{j\in[n]}, z)$

Procedure $\text{H}(\mathbf{pk}, R', m, \beta)$:
// Denote the random oracle table by H
36 **return** $H[\mathbf{pk}, R', m, \beta]$ if $H[\mathbf{pk}, R', m, \beta] \neq \perp$
37 $vid := vid + 1; V := V \cup \{vid\}$
38 $c := \mathbf{h}_{vid}; c' := c + \beta; H[\mathbf{pk}, R', m, \beta] := c'$
39 **return** $(vid, H[\mathbf{pk}, R', m, \beta])$

[a] $\mathbf{c}_{sid|\mathbf{pk}}$ denotes the sub-vector of $\mathbf{c}_{sid}$ consisting of the components indexed by the queried ring $\mathbf{pk}$.

Fig. 10: The Wrapping Adversaries $\mathcal{A}'_\ell$ for $\ell \in [q+1]$ in $\mathsf{BRS}_{\mathsf{HE}}$

Adversary $\mathcal{B}(\text{pp})$:
00 $i^* \leftarrow_\$ [q+1]$
01 $\mathbf{h} \leftarrow_\$ \mathcal{S}^{q_{\text{H}}}$
02 $\omega \leftarrow_\$ \Omega$
03 $\{sk_j\}_{j\in[N]} \leftarrow_\$ \mathcal{D}^N$
04 $(\hat{\mathbf{J}}_{i^*}, \hat{\boldsymbol{\chi}}_{i^*}) := \mathcal{A}'_{i^*}(I = (\text{pp}, \{sk_j\}_{j\in[N]}), \mathbf{h}; \omega)$
05 **return** $\perp$ if $\hat{\mathbf{J}}_{i^*} = 0$
06 $\mathbf{h}'_{[\hat{\mathbf{J}}_{i^*}:]} \leftarrow_\$ \mathcal{S}^{q_{\text{H}} - \hat{\mathbf{J}}_{i^*} + 1}; \mathbf{h}' := \mathbf{h}_{[:\hat{\mathbf{J}}_{i^*}-1]} || \mathbf{h}'_{[\hat{\mathbf{J}}_{i^*}:]}$
07 $(\hat{\mathbf{J}}'_{i^*}, \hat{\boldsymbol{\chi}}'_{i^*}) := \mathcal{A}'_{i^*}(I = (\text{pp}, \{sk_j\}_{j\in[N]}), \mathbf{h}'; \omega)$
08 **return** $(\hat{\boldsymbol{\chi}}_{i^*}, \hat{\boldsymbol{\chi}}'_{i^*})$ if $\hat{\boldsymbol{\chi}}_{i^*} \neq \hat{\boldsymbol{\chi}}'_{i^*}$
09 **return** $\perp$

Fig. 11: Adversary $\mathcal{B}$ against the Collision Resistance of LF

Signer-anonymity. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be an adversary in the signer-anonymity experiment. $\mathcal{A}_1$ takes as input $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and $\mathbf{pk} := \{pk_j\}_{j\in[N]}$, where $(sk_j, pk_j) := \text{KeyGen}(\text{pp}; r_j)$ for all $j \in [N]$. Unless otherwise stated, the random oracle is simulated honestly via lazy sampling. $\mathcal{A}_1$ queries the signing oracle pair $(\mathcal{O}_{\text{Sign}_1}, \mathcal{O}_{\text{Sign}_2})$ (simulated honestly), outputs $(i_0, i_1, \mathbf{pk}^* = \{pk_j\}_{j\in[n]})$, and passes the internal state state to $\mathcal{A}_2$. $\mathcal{A}_2$ in turn executes with the challenge oracles $(\mathcal{O}^{sess,b}_{\text{SignLoR}_1}, \mathcal{O}^{sess,b}_{\text{SignLoR}_2})$, obtaining a transcript $T_b := (R_b, ch, \{c'_j\}_{j\in[n]}, z_b)$, where $ch := (ek, \{\text{ct}(\beta_j)\}_{j\in[n]}, \text{ct}(c'))$ is adaptively chosen by $\mathcal{A}$ based on R_b. Since $\mathcal{A}$ holds the key pair (ek, dk) for the HE scheme, we simplify rewrite the adversary submitted challenge to its plaintext form, *i.e.*, $ch = (\{\beta_j\}_{j\in[n]}, c')$. Note that we do not require $\mathcal{O}_{\text{SignLoR}_2}$ to verify that the sum $\sum_{j\in[n]} \beta_j$ matches the β used in the random oracle query to generate c' (*i.e.*, $c' = \text{H}(\mathbf{pk}^*, R', m, \beta) + \beta$). That is, $\mathcal{A}$ is allowed to submit malformed challenge to $\mathcal{O}_{\text{SignLoR}_2}$.

Now, we examine the transcript elements.

$$R_b = \mathsf{LF.F}(r) + \sum_{j\in[n]\wedge j\neq i_b} c_j \cdot pk_j,$$

$$c'_j = c_j + \beta_j \ (\forall j \in [n] \wedge j \neq i_b),\ c'_{i_b} = c' - \sum_{j\in[n]\wedge j\neq i_b} c'_j,$$

$$z_b = r - (c'_{i_b} - \beta_{i_b}) \cdot sk_{i_b}$$

where $r \leftarrow\!\!\$\ \mathcal{D}$ and $c_j \leftarrow\!\!\$\ \mathcal{S}, \forall j \in [n] \wedge j \neq i_b$. Denote two implicit values $c_{i_b} = c'_{i_b} - \beta_{i_b}$ and $c = c' - \sum_{j\in[n]} \beta_j$. In either experiment ($b = 0$ or $b = 1$), the set $\{c_j\}_{j\in[n]}$ consists of $n-1$ uniformly sampled elements from $\mathcal{S}$, with the remaining c_{i_b} determined by $c_{i_b} = c - \sum_{j\in[n]\wedge j\neq i_b} c_j$. Since the sum constraint is symmetric, the joint distribution of the set $\{c_j\}_{j\in[n]}$ is identical in both experiments.

Moreover, as r is sampled uniformly from $\mathcal{D}$, the response z_b is uniformly distributed in $\mathcal{D}$. A similar argument applies to R_b. Even if $\mathcal{A}$ possesses the secret keys (sk_{i_0}, sk_{i_1}), it cannot distinguish b because the transcript is consistent with either key. Specifically, for either index $k \in \{i_0, i_1\}$, there exists a valid randomness r_k *s.t.* the transcript (particularly (R_b, z_b)) can be explained as a valid pair generated by k using r_k:

$$r_k := z_b + c_k \cdot sk_k, \text{ and} \tag{B.1}$$

$$\begin{aligned} R_b &= \mathsf{LF.F}(r_k) + \sum_{j\in[n]\wedge j\neq k} c_j \cdot pk_j \\ &= \mathsf{LF.F}(z_b) + c_k \cdot pk_k + \sum_{j\in[n]\wedge j\neq k} c_j \cdot pk_j \qquad (B.2) \\ &= \mathsf{LF.F}(z_b) + \sum_{j\in[n]} c_j \cdot pk_j. \qquad (B.3) \end{aligned}$$

Therefore, the distribution of the full transcript is identical for $b = 0$ and $b = 1$, providing anonymity even under full key exposure.

Blindness. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be an adversary in the blindness experiment. $\mathcal{A}_1$ takes as input $\mathrm{pp} \leftarrow \mathrm{Setup}(1^\lambda)$ and $(sk_j, pk_j) \leftarrow \mathrm{KeyGen}(\mathrm{pp})$ for all $j \in [n]$, where the (semi-)honest signer model ensures $pk_j = \mathsf{LF.F}(sk_j)$. After the execution of $\mathcal{A}_2$ with the challenge oracles $(\mathcal{O}_{\mathrm{Init}}, \mathcal{O}_{\mathrm{User}_1}, \mathcal{O}_{\mathrm{User}_2})$ (suppose, *w.l.o.g.*, it invokes the challenge oracles on behalf of signer index i), $\mathcal{A}$ holds two valid outputs $(\mathbf{pk}, m_0, \sigma_0)$ and $(\mathbf{pk}, m_1, \sigma_1)$ (otherwise, $\mathcal{A}$ obtains $\sigma_0 = \sigma_1 = \perp$, and the advantage is defined as 0). $\mathcal{A}$ also learns two transcripts from the interaction, denoted by $T^{(k)} = (R^{(k)}, ch^{(k)}, z^{(k)})$ for $k \in \{0, 1\}$. The goal of $\mathcal{A}$ is to match the ring-message-signature tuple to the corresponding transcript.

We expand $ch^{(k)} = (ek^{(k)}, \{\mathrm{ct}(\beta_j^{(k)})\}_{j\in[n]}, \mathrm{ct}(c'^{(k)}))$ and $\sigma_b = (\{c'_{j,b}\}_{j\in[n]}, z'_b, \beta_b)$, where $\beta_b = \sum_{j\in[n]} \beta_j^{(k)}$. The adversary can extract $\alpha_b^{(k)} = z'_b - z^{(k)}$ and, from the reconstruction of $R'_b = \mathsf{LF.F}(z'_b) + \sum_{j\in[n]} c'_{j,b} \cdot pk_j$, learn $c_b := \mathrm{H}(\mathbf{pk}, R'_b, m_b, \beta_b)$. Additionally, since the adversary knows $\{c_j^{(k)}\}_{j\in[n]\wedge j\neq i}$ and $\mathrm{ct}(c_i^{(k)}) = \mathrm{ct}(c^{(k)}) \ominus \mathrm{ct}(\beta_i^{(k)})$ (cf. Equation 3.1) , we must show that the encrypted blinding factors and the challenge grant $\mathcal{A}$ no advantage in linking:

$$\bigoplus_{j\in[n]} \mathrm{ct}(\beta_j^{(k)}) \text{ to } \beta_b, \text{ or}$$

$$\{\mathrm{ct}(c_j'^{(k)}) = \mathrm{ct}(c_j^{(k)}) \oplus \mathrm{ct}(\beta_j^{(k)})\}_{j\in[n]} \text{ to } \{c'_{j,b}\}_{j\in[n]}$$

Here, we define a sequence of games that differ only in the way $(\{\mathrm{ct}(\beta_j^{(k)})\}_{j\in[n]}, \mathrm{ct}(c'^{(k)}))$ of each transcript $T^{(k)}$ is generated.

- Game_0. This is the real blindness experiment for $\mathsf{BRS}_{\mathsf{HE}}$.
- Game_1. For each $b \in \{0, 1\}$ and the corresponding session, we modify Game_0 by having the challenge oracles sample $\hat{\beta}_{j,b} \leftarrow\!\!\$\ \mathcal{S}, \forall j \in [n] \wedge j \neq i$, set $\hat{\beta}_{i,b} := \beta_b - \sum_{j\in[n]\wedge j\neq i} \hat{\beta}_{j,b}$ and set $\hat{c_b}' := \sum_{j\in[n]} c'_{j,b}$. The other parts of the experiment are unchanged, as the blinding factor and the challenge remain identical across the games. By HE's semantic security, the ciphertexts $(\{\mathrm{ct}(\beta_j)^{(k)}\}_{j\in[n]}, \mathrm{ct}(c'^{(k)}))$ (for both $k\in\{0, 1\}$) and $(\{\mathrm{ct}(\hat{\beta}_{j,b})\}_{j\in[n]}, \mathrm{ct}(\hat{c_b}'))$ (for both $b \in \{0, 1\}$) are computationally indistinguishable to $\mathcal{A}$. Therefore, it suffices to prove blindness in Game_1, where the simulated $(\{\hat{\beta}_{j,b}\}_{j\in[n]}, \hat{c}'_b)$ (for $b \in \{0, 1\}$) are uniformly distributed and derived solely from σ_b, and are hence independent of session index k. Moreover, they remains independent from c_b as long as $\mathcal{A}$ has not made the random oracle query at $(\mathbf{pk}, R'_b, m_b, \beta_b)$.

Similarly, the uniformity of $\alpha_b^{(k)}$ is implied by the uniformity of $z'_b = z_b + \alpha_b$, which comes from the experiment. Note that α_b also remains independent from c_b as long as $\mathcal{A}$ has not made the random oracle query at $(\mathbf{pk}, R'_b, m_b, \beta_b)$.

Now, let BE be the event that $\mathcal{A}$ queries the random oracle at $(\mathbf{pk}, R'_b, m_b, \beta_b)$ during some session k. Since β_b is uniformly distributed, it is unpredictable to $\mathcal{A}$ at query time (before σ_b is revealed). The probability $\Pr[\mathrm{BE}] \leq q_{\mathrm{H}}/|\mathcal{S}|$ is bounded by a negligible function in λ. The remaining randomness, the key pair $(ek^{(k)}, dk^{(k)})$, is also sampled independently and identically across the two sessions for both messages. It follows that the joint distribution of the two transcripts together with the two output signatures is *exchangeable* under swapping the session labels $k = 0$ and $k = 1$. Therefore, accounting for $\Pr[\mathrm{BE}]$ and the computational indistinguishability implied by the semantic security of HE, $\mathcal{A}$ has only negligible advantage in the blindness experiment of $\mathsf{BRS}_{\mathsf{HE}}$. □

B.2 Proof of Theorem 2

Proof. We recall the statement as follows. Let LF be an LF family, H be a hash function modeled as a random oracle, COM be a commitment scheme, and Π_{CPSA} be a NIZKAoK. The construction $\mathsf{BRS}_{\mathsf{CP}}$ in Figure 3 satisfies the following properties in the random oracle model: correctness (Definition 10); $(k, k+1)$-unforgeability *w.r.t.* insider corruption (Definition 11) if LF is collision resistant, COM is binding, and Π_{CPSA} is knowledge sound; signer-anonymity against full key exposure (Definition 12); and statistical blindness if COM is perfect hiding and Π_{CPSA} is statistical zero-knowledge.

Correctness. The correctness of $\mathsf{BRS}_{\mathsf{CP}}$ follows straightforwardly from the completeness of Π_{CPSA} for the relation in Equation 3.4.

Unforgeability. The proof proceeds analogously to the unforgeability of $\mathsf{BRS}_{\mathsf{HE}}$, employing a sequence of wrapping adversaries $\mathcal{A}'_\ell, \forall \ell \in [q+1]$ invoked by the reduction adversary $\mathcal{B}$ to break the collision resistance of LF. We define $\mathcal{A}'_\ell$ in Figure 12; whereas, $\mathcal{B}$ is identical to the one used in $\mathsf{BRS}_{\mathsf{HE}}$ and is referenced in 11.

Again, an execution of $\mathcal{A}'_\ell$ is fixed by $I = (\mathrm{pp}, \{sk_j\}_{j\in[N]})$, the vector $\mathbf{h}$, and the randomness ω. The set of successful inputs $\mathcal{W}$ for $\mathcal{A}'_\ell$ is defined as $\mathcal{W} := \left\{(I, \omega, \mathbf{h}) \mid \hat{\mathbf{J}}_\ell \neq 0; (\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell) := \mathcal{A}'_\ell(I, \mathbf{h}; \omega)\right\}$.

Unlike the $\mathsf{BRS}_{\mathsf{HE}}$ case, here we must close the gap incurred by the extraction failure of Π_{CPSA}'s extractor E (Line 12), which is of negligible probability by the knowledge soundness of Π_{CPSA} (Definition 9). Consequently, the probability of a tuple belonging to $\mathcal{W}$, *i.e.* $(I, \omega, \mathbf{h}) \in \mathcal{W}$ for $pp \leftarrow \mathrm{Setup}(1^\lambda)$ and $(\{sk_j\}_{j\in[N]}, \omega, \mathbf{h}) \leftarrow\!\!\$\ \mathcal{D}^N \times \Omega \times \mathcal{S}^{q_{\mathrm{H}}}$, is at least the advantage of $\mathcal{A}$ against the unforgeability of $\mathsf{BRS}_{\mathsf{CP}}$ (*i.e.*, the probability formula in Definition 11) minus a negligible term corresponding to the soundness error.

Regarding $\mathcal{B}$, the linearity of the summation $c' = \sum_{j\in[n} c_j$ is addressed identically to the $\mathsf{BRS}_{\mathsf{HE}}$ analysis, specifically via the inclusion of $\boldsymbol{\chi}_{i^*}[0]$. A distinct issue arises in $\mathsf{BRS}_{\mathsf{CP}}$: if the difference between the values $\boldsymbol{\chi}_{i^*}$ extracted from two runs of $\mathcal{A}'_{i^*}$ manifests solely in the opening of R_β, *i.e.* yielding two valid but distinct pairs (β, O_β) and (β', O'_β). In this scenario, $\mathcal{B}$ fails to extract a meaningful collision for LF. However, such an event implies breaking the binding property of the commitment scheme COM; thus, it occurs with negligible probability and incurs a negligible loss in $\mathcal{B}$'s advantage.

Signer-anonymity. The signer-anonymity of $\mathsf{BRS}_{\mathsf{CP}}$ follows identically to that of $\mathsf{BRS}_{\mathsf{HE}}$. The key observation is that, regardless of whether the deterministically chosen pk_l coincides with i_0 or i_1, the linear relation governing the challenges $\{c_j\}_{j\in[n]}$ ensures that the signer commitment-response pair (R, z) can always be consistently explained by both sk_{i_0} and sk_{i_1} (cf. Equation B.1). This consistency leads to an identical result from the random oracle query in either case, rendering the transcripts indistinguishable.

Blindness. We consider the same setting as the proof of blindness for $\mathsf{BRS}_{\mathsf{HE}}$. Concretely, for the signature on message m_b, we recall in $\mathsf{BRS}_{\mathsf{CP}}$ that $\sigma_b = (R'_b, R_{\beta,b}, z'_b, \pi_b)$, and for the transcript of session $k \in \{0,1\}$, $T^{(k)} = (R^{(k)}, c'^{(k)}, z^{(k)})$.

We define the following sequence of games.

- Game_0. This is the real blindness experiment for $\mathsf{BRS}_{\mathsf{CP}}$.
- Game_1. For each $b \in \{0,1\}$ and the corresponding session, we modify Game_0 by having the challenge oracles output the simulated $\hat{\pi}_b \leftarrow \Pi_{\mathsf{CPSA}}.\mathrm{Sim}(\mathrm{crs}, \tau, x_b = (\mathbf{pk}, R'_b, R_{\beta,b}, z'_b))$ instead of π_b. By the zero-knowledge property of Π_{CPSA}, $\hat{\pi}_b$ distributes (statistically) close to π_b and is independent of session index k.

Note that the adversary can extract $\alpha_b^{(k)} = z'_b - z^{(k)}$ and compute $c_b = \mathrm{H}(\mathbf{pk}, R'_b, m_b, R_{\beta,b})$, hence learning $\beta_b^{(k)} = c'^{(k)} - c_b$. Since the commitment scheme satisfies perfect hiding, $R_{\beta,b}$ distributes identically across the two sessions *w.r.t.* $\beta_b^{(0)}$ and $\beta_b^{(1)}$.

Adversary $\mathcal{A}'_\ell(I = (\mathrm{pp}, \{sk_j\}_{j\in[N]}), \mathbf{h}; \omega)$:

00 Parse $(\mathbf{r}, \mathbf{c})$ from ω
01 $\mathbf{pk} := \{\mathsf{LF.F}(sk_j)\}_{j\in[N]}$
02 $sid, vid, q, ctr := 0; S, V := \emptyset$
03 $(\mathbf{pk}^*, \mathbf{m}^*, \boldsymbol{\sigma}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{Corrupt}}, \mathcal{O}_{\mathrm{Sign}_1}, \mathcal{O}_{\mathrm{Sign}_2}, \mathrm{H}}(\mathbf{pk})$
04 **return** $\perp$ if $\mathbf{pk}^* \not\subseteq \mathbf{pk} \setminus C$
05 $\mathbf{R}, \mathbf{R}', \mathbf{c}', \mathbf{z} := \emptyset$
06 For $l \in [k+1]$ do:
07 Parse $\boldsymbol{\sigma}^*_l = (R', R_\beta, z', \pi)$
08 $\mathbf{R}'_l = (\mathbf{R}'_l[0], \mathbf{R}'_l[1]) := (R', R_\beta)$
09 $(vid, c') := \mathrm{H}(\mathbf{pk}^*, \mathbf{R}'_l[0], \mathbf{m}^*_l, \mathbf{R}'_l[1])$
10 **return** $\perp$ if $vid \notin V$
11 If $\Pi_{\mathsf{CPSA}}.\mathrm{Verify}((\mathbf{pk}^*, \mathbf{R}'_l, z'), \pi) = 1$:
12 $(\{c_j\}_{j\in[n]}, \beta, O_\beta) \leftarrow \Pi_{\mathsf{CPSA}}.\mathrm{E}^{\mathcal{A}}(\mathrm{pp})$
13 $ctr := ctr + 1$
14 $\hat{\mathbf{z}}'_{ctr} := z'$
15 $\hat{\mathbf{h}}_{ctr} := \mathbf{h}_{vid}$
16 $\hat{\boldsymbol{\chi}}_{ctr} = (\hat{\boldsymbol{\chi}}_{ctr}[0], \hat{\boldsymbol{\chi}}_{ctr}[1])$, where:
17 $\hat{\boldsymbol{\chi}}_{ctr}[0] := (\{c_j\}_{j\in[n]}, \beta, O_\beta)$
18 $\hat{\boldsymbol{\chi}}_{ctr}[1] := \hat{\mathbf{z}}'_{ctr} + \sum_{j\in[n]} c_j \cdot sk_j$
19 $\hat{\mathbf{J}}_{ctr} := vid$
20 $V := V \setminus \{vid\}$
21 $q_{\mathrm{H}} := vid$
22 **return** $(\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell)$ if $ctr \geq q+1$
23 **return** $(\hat{\mathbf{J}}_\ell, \hat{\boldsymbol{\chi}}_\ell) := (0, 0)$

Procedure $\mathcal{O}^{C}_{\mathrm{Corrupt}}(i)$:

24 **return** sk_i

Procedure $\mathcal{O}^{sid,S}_{\mathrm{Sign}_1}(i, \mathbf{pk})$:

25 **return** $\perp$ if $pk_i \neq \mathbf{pk}$
26 $sid := sid + 1; S := S \cup \{sid\}$
27 $\mathbf{c}'_{sid}, \mathbf{z}_{sid} := \perp$
28 [a] $\mathbf{R}_{sid} := \mathsf{LF.F}(\mathbf{r}_{sid}) + \langle \mathbf{c}_{sid|\mathbf{pk}}, \mathbf{pk} \rangle$
29 **return** $(sid, \mathbf{R}_{sid})$

Procedure $\mathcal{O}^{S,q}_{\mathrm{Sign}_2}(sid, ch)$:

30 **return** $\perp$ if $sid \notin S$
31 Parse $(i, \mathbf{pk})$ from state_{sid} inherited from $\mathcal{O}_{\mathrm{Sign}_1}$
32 Parse $\mathbf{c}_{sid|\mathbf{pk}} = \{c_j\}_{j\in[n]}$ and $ch = c'$; $\mathbf{c}'_{sid} := c'$
33 $c_i := c' - \sum_{j\in[n]\wedge j\neq i} c_j$
34 $z := \mathbf{r}_{sid} - c_i \cdot sk_i$
35 $S := S \setminus \{sid\}; q := q + 1$
36 **return** $\mathbf{z}_{sid} := (\{c_j\}_{j\in[n]}, z)$

Procedure $\mathrm{H}(\mathbf{pk}, R', m, R_\beta)$:

// Denote the random oracle table by H
37 **return** $H[\mathbf{pk}, R', m, R_\beta]$ if $H[\mathbf{pk}, R', m, R_\beta] \neq \perp$
38 $vid := vid + 1; V := V \cup \{vid\}$
39 $c' := \mathbf{h}_{vid}; H[\mathbf{pk}, R', m, R_\beta] := c'$
40 **return** $(vid, H[\mathbf{pk}, R', m, R_\beta])$

[a] $\mathbf{c}_{sid|\mathbf{pk}}$ denotes the sub-vector of $\mathbf{c}_{sid}$ consisting of the components indexed by the queried ring $\mathbf{pk}$.

Fig. 12: The Wrapping Adversaries $\mathcal{A}'_\ell$ for $\ell \in [q+1]$ in $\mathsf{BRS_{CP}}$

The remainder of the blindness analysis proceeds identically to that of [29, Theorem 5.8] and $\mathsf{BRS_{HE}}$. Specifically, the blinding factors $\alpha_b^{(k)}$ and $\beta_b^{(k)}$ are uniformly distributed and independent of $c_b = \mathrm{H}(\mathbf{pk}, R'_b, m_b, R_{\beta'})$, provided the adversary has not queried the random oracle on the corresponding input (a condition analogous to the "bad event" in the blindness analysis of $\mathsf{BRS_{HE}}$). Combining all pieces above yields the blindness of $\mathsf{BRS_{CP}}$. □